

Easy Returns – Aplicație Desktop PENTRU GESTIONAREA DECONTURILOR SI ORDINELOR DE DEPLASARE DINTR-O FIRMA

Candidat: Mirulescu George-Remus
Coordonator științific: Conf.dr.ing. Alexandru, Topîrceanu

Sesiunea: Iunie 2023

CUPRINS

1. INTRODUCERE	4
1.1 Context	4
1.2 Problema din perspectiva firmelor	4
1.3 Cum rezolvă Easy Returns problema	4
1.4 Cum funcționează	5
1.5 Cu ce vă ajută Easy Returns	5
1.6 Comparație cu alte aplicații existente	6
2. TEHNOLOGII FOLOSITE	8
2.1 React	8
2.2 Material UI	8
2.3 Web Socket	8
2.4 Axios	9
2.5 Redux	9
2.6 Node.js	9
2.7 Amazon Web Services(AWS)	9
2.8 Azure AD	10
2.9 Microsoft Graph API	10
2.10 Express.js	10
2.11 Mindee OCR	10
2.12 Sequelize	10
2.13 MySQL	11
2.14 Docker	11
2.15 ESLint	12
2.16 Github Actions	12
2.17 Terraform	13
2.18 Versionarea Codului	13
2.19 Husky	14
3. Proiectare	15
3.1 Proiectarea Aplicației	15

3.2 Diagrama UML a funcționalităților aplicației	16
3.3 Navigarea în aplicație	18
3.4 Proiectarea bazei de date	19
4. Implementarea Aplicației	27
4.1 Implementarea autentificării folosind SSO Login cu OAuth2.0	27
4.2 Implementarea funcționalității de scanare a unui bon fiscal folosind un algoritm de OCR(Optical character recognition)	31
4.3 Implementarea și folosirea containerelor de Docker	34
4.4 Conexiunea cu baza de date	38
4.5 Crearea tabelor folosind migrari	40
4.6 Implementarea unui debounce search pentru lista de angajati.....	42
4.7 Implementarea exportului CSV pentru deconturi	45
5. Utilizarea aplicației.....	48
5.1 Ecranul de login.....	48
5.2 Ecranul cu ordinele de deplasare	48
5.3 Ecranul în care putem crea un ordin de deplasare	49
5.4 Ecranul cu deconturi	50
5.5 Ecranul în care putem crea un decont	51
5.6 Ecranul administratorilor în care se poate adăuga contul bancar pentru fiecare salariat	53
5.7 Ecranul de setări.....	53
6. Concluzii	56
6.1 Concluzii Generale	56
6.2 Funcționalități viitoare	56

1. INTRODUCERE

1.1 Context

În momentul actual, firmele din Romania se confruntă cu o birocrație îngrozitoare. Din prima până în ultima zi de activare, firmele se confruntă cu o birocrație enormă pentru cereri, deconturi, declarații, copii de CUI și CI, foi de parcurs etc. Întrebând mai mulți directori, am realizat că majoritatea firmelor își realizează deconturile în maniera clasică: directorul oferă un model în excel pe care fiecare angajat îl completează, iar apoi îl trimite înapoi directorului care primește deconturi pe mail, pe telefonul mobil, pe Microsoft Teams, pe whatsapp etc, iar pentru fiecare este nevoit să printeze foi pentru actele asociate decontului, să le completeze de mână, să semeneze, să le scaneze, să le încarce din nou pe calculator, să țină gestiunea(cine ce decont a făcut), iar acest proces îndelungat continuă. Analizând problema, am creat aplicația Easy Returns care vine în sprijinul firmelor și le ajută să gestioneze deconturile și ordinele de deplasare.

1.2 Problema din perspectiva firmelor

Vorbind cu directorul firmei la care lucrez și observând că acesta petrece multe ore realizând actele pentru deconturi, am realizat că este o problema. Este greu că un om să facă față deconturilor dintr-o firma chiar și dacă are puțini angajați, pentru că fiecare dintre ei pot face mai multe deconturi în fiecare luna, iar datorită birocrației lungi se poate ajunge ușor la sute de foi de printat, completat și semnat. Pe lângă asta, din cauză că deconturile veneau din toate directile, fie email, fie teams etc, era greu de reținut locul lor de proveniență, iar de multe ori se uita de ele angajatul fiind nevoit să îi reamintească de decont. Pe lângă asta, în multe zile, directorul, rămânea peste program doar că să aibă timp să realizeze actele de deconturi pentru fiecare angajat.

Concluzia la care am ajuns este că soluția clasică abordată în foarte multe firme, de a se ocupa de deconturi în aceasta maniera care consuma foarte mult timp și energie nu este deloc eficientă mai ales în firmele marii cu mulți angajați unde poți petrece zile întregi până termini cu birocrația. Din această cauză, am ajuns să realizez aplicația de deconturi care face totul pentru tine, îmbunătățind astfel sistemul de gestiune al deconturilor din firme.

1.3 Cum rezolvă Easy Returns problema

Aplicația este gândită să fie un sediu central în care toți angajații din firma pot intra să își deconteze diverse bonuri fiscale pentru abonamente la sala, cursuri, ieșiri cu firma, ordine de deplasare și orice altceva. Aplicația poate fi accesată foarte ușor de către orice angajat deoarece fiecare dintre ei primește un cont(de microsoft) pentru a începe treaba, iar folosind SSO de la Azure automat contul este înregistrat în aplicație. În acest mod, angajatul nu are nici o bătaie de cap pentru a se înregistra. Aplicația ajută firmele să își gestioneze deconturile. Fiecare angajat poate crea deconturi, iar unui decont i se poate asocia unul sau mai multe bonuri care urmează să

fie decontat/e. De asemenea, este posibilă și crearea unui ordin de deplasare care mai apoi poate fi asociat unui decont. Astfel, bonurile fiscale obținute în timpul ordinului de deplasare pot fi asociate cu acesta, iar ținând cont de diurna, aplicația îi spune angajatului câți bani mai trebuie să primească/restituie. Angajații au posibilitatea să vizualizeze fiecare ordin de deplasare împreună cu bonurile fiscale sau doar decontul în cazul în care acesta nu are asociat un ordin de deplasare. În cele din urmă, directorul poate intra pe aplicație și să vadă toate deconturile, pozele cu bonuri fiscale asociate lor, suma totală de restituit, suma fiecărui bon fiscal, furnizorul, poate modifica deconturile, accepta/refuza/șterge un decont sau ordin de deplasare, poate semna decontul direct din aplicație, poate descărca toate actele necesare pentru un decont și poate asocia fiecărui angajat un cont bancar, în acest mod înlocuind tot procesul îndelungat de procesare a cererilor de deconturi. De asemenea, un utilizator mai poate să își schimbe poza de profil, poate selecta o tematică light sau dark a aplicației în funcție de preferințe, poate filtra rezultatele de deconturi și ordinele de deplasare în cazul în care sunt prea multe, poate ordona descrescător sau crescător rezultatele după orice coloană, își poate vizualiza rolul (ADMIN sau STANDARD USER). În plus, atunci când un decont sau un ordin de deplasare este creat, administratorul este notificat prin email, precum și în aplicație că există noi deconturi care necesită să fie revizuite, iar după ce acesta ia o decizie și acceptă sau refuză cererea, angajatul este, de asemenea, notificat atât prin mail cât și în aplicație de decizia luată.

1.4 Cum funcționează

- Utilizatorii sunt înregistrați automat cu contul de microsoft de lucru
- Salariații adaugă sumele cheltuite și chitanțele
- Aprobatorii aprobă, modifică, semnează cheltuielile și verifică deconturile
- Administratorii emit dispoziții de încasare sau de plată și schimbă statusul deconturilor

1.5 Cu ce vă ajută Easy Returns

- Aveți situația clară a deconturilor și a sumelor datorate angajaților
- Ordinele de deplasare și deconturile sunt completate rapid
- Actele asociate cheltuielilor sunt generate automat la o simplă apăsare de buton
- Aplicația poate fi folosită ușor din orice browser fără să fie nevoie să instalați nimic
- Accesați aplicația de pe orice telefon, calculator sau tabletă
- Aveți în orice moment situația deconturilor și a cheltuielilor pentru fiecare salariat
- Aplicația vă ajută să economisiți timp și să aveți un mod de lucru organizat pentru a gestiona deconturile de cheltuieli
- Diurnele acordate sunt calculate automat în funcție de durata deplasării și de țara în care se face deplasarea.
- Puteți să configurați diurnele acordate pentru fiecare țară.

1.6 Comparație cu alte aplicații existente

Pentru a scoate în evidența principalele funcționalități ale aplicației am ales să fac o comparație între aceasta și alte aplicații care se afla în momentul de față pe piață. Așadar, doua aplicații asemănătoare celei dezvoltate sunt **ScoreExpense** și **Iconta**. Tabelul 1.1 de mai jos reprezintă o analiza a funcționalităților celor doua aplicații comparativ cu Easy Returns.

Caracteristici si functionalitati	SocrateExpense	Iconta.ro	Easy Returns
Link site	<u>SocrateExpense</u>	<u>Iconta</u>	<u>Easy Returns</u>
Aplicatie web	x	x	x
Aplicatie mobile	x	-	-
User login	x	x	x
Update în timp real	x	x	x
Incarcare poze	x	x	x
Descarcare PDF	x	x	x
Scanare OCR poza	-	-	x
Generare Acte Decont	x	x	x
Selectare moneda	x	x	x
Dark mode	-	-	x
Configurare diruna in functie de tara	?	x	x
Notificari	-	-	x
Configurare conturi bancare salariat	-	-	x
SSO Login	-	-	x
Poza Profil	-	-	x

Tabela 1.1: Analiză comparativă între SocrateExpense, Iconta și Easy Returns

Din tabelul comparativ, se observă că Easy Returns are funcționalități în plus față de cele două aplicații, singurul dezavantaj în comparație cu SocrateExpense fiind că nu este și o aplicație mobilă. Easy Returns se remarcă prin următoarele funcționalități:

1. Un algoritm de scanare a bonului fiscal, astfel utilizatorul nu trebuie să introducă de mana date precum data decontului, suma, furnizorul, ci poate încarca o poză cu bonul fiscal, iar datele necesare decontului sunt automat extrase de pe bon folosind un algoritm de inteligență artificială de recunoaștere a caracterelor.

2. Utilizatorul poate schimba tema de culoare pe care o are aplicația în funcție de preferință, acesta poate alege să activeze dark mode în aplicație. Această funcționalitate a fost gândită deoarece o temă mai închisă la culoare reduce oboseala ochilor noaptea, iar textul este mai ușor de citit.

3. Un sistem de notificări bazat pe sochet astfel încât fiecare utilizator este anunțat atunci când administratorul a acceptat sau a refuzat cererea de cheltuieli. Fiecare utilizator primește un prim email în care își da aprobarea că poate primi notificări din partea Easy Returns, iar apoi va

primii notificări pe mail sau atunci când intra în aplicație. De asemenea, administratorul este anunțat prin email atunci când un utilizator creează un decont.

4. Fiecare utilizator își poate adauga o poza de profil care va fi vizibilă în bara de navigare.

5. Single sign-on login care este un serviciu de autentificare ce permite unui utilizator să folosească aceleași credențiale pentru a accesa mai multe aplicații. În acest mod fiecare angajat dintr-o firmă vă putea accesa aplicația folosind adresa de email de la lucru și nu o să își mai bata capul să își creeze un cont.

6. Configurarea de conturi bancare pentru fiecare salariat poate fi făcută de către administrator. După care, se poate descărca un fișier CSV cu toți anagajații și conturile lor bancare care ajută admin-ul să știe în ce conturi trebuie să facă plata asociată deconturilor.

2. TEHNOLOGII FOLOSITE

2.1 React

React este un framework folosit pentru a construi interfețe cu utilizatorul care se bazează pe componente și este întreținut de către Meta. ReactJs împarte interfață în piese reutilizabile de cod numite componente. Componentele din React funcționează similar cu funcțiile din javascript și typescript pentru că acceptă un număr arbitrar de proprietăți. Este posibil să avem câte componente dorim astfel încât codul să fie mai organizat, iar ideea de baza este să avem componente reutilizabile ca să nu duplicăm cod. React este folosit pentru a crea SPA(single page applications) care sunt afișate ca o pagina statica. Acest framework folosește Virtual DOM ca și strategie pentru a avea cat mai puține operații în DOM atunci când se rerendeaza interfața cu utilizatorul. Un Virtual DOM este același lucru cu un DOM real însă este o copie mai mica care nu poate manipula elementele de pe ecran. Atunci când o componenta se schimbă, React creează un nou arbore cu nodul updatat și astfel rezultă un virtual DOM updatat care cu ajutorul unei librării numita ReactDOM vă fi comparat cu o copie, iar în acest mod se modifică doar nodul modificat.

2.2 Material UI

Material UI este o librărie open-source de componente pentru React care implementează material design de la Google. Material UI include o colecție cuprinzătoare de componente gata făcute și pregătite să fie utilizate. De asemenea, librăria poate fi folosită pentru a crea componente custom complexe, prin combinarea componentelor deja existente, în cazul în care componentele oferite nu sunt suficiente. Avantajele care vin la pachet cu aceasta librărie sunt: peste 2500 de contribuitori care au participat la dezvoltare pentru a oferi soluții pe partea de UI, componentele folosite arată bine din prima fără să mai fie nevoie să scriem fișiere întregi de CSS, iar componentele sunt responsive în mare parte așadar ne ușurează munca atunci când vrem ca aplicația să fie responsive pentru mobil.

2.3 Web Socket

Aplicația folosește un web socket special construit pentru sistemul de notificări. Spre deosebire de HTTP care este un protocol unidirecțional în care un client trimite o cerere, iar serverul răspunde, un web socket este bidirecțional folosind un protocol full-duplex care este folosit pentru comunicarea între client și server. Conexiunea dintre client și server vă fi menținută până când este terminată de una dintre cele doua părți(client sau server). După ce conexiunea este închisă de client sau de server, aceasta se termină la ambele capete.

2.4 Axios

Axios este o librărie HTTP care ne permite să consumăm un API. Librăria oferă mai multe posibilități prin care putem face cereri HTTP precum GET, PUT, POST și DELETE și poate fi folosită în orice framework javascript. Librăria este folosită deoarece funcționează bine cu date JSON în sensul că de multe ori nici nu mai este necesar să setăm headerele, în plus funcțiile sunt foarte sugestive adică avem funcții denumite „get”, „post”, „put”, „delete” pentru fiecare operație HTTP. Principalele avantaje față de fetch sunt că axios poate fi configurat adică putem seta un url de baza, requesturile pot fi interceptate și putem seta ce header dorim. Axios serializează automat datele pentru multipart/form-data și x-www-form-urlencoded, face transformări automate pentru datele de tip JSON și oferă o protecție pe partea de client împotriva atacurilor CSRF(Cross-Site Request Forgery).

2.5 Redux

Redux este o librărie folosită pentru managementul de stări globale. Principalul avantaj este că o variabilă poate fi accesată în orice componentă copil, tot ceea ce trebuie să facem este să înfășurăm componentele copil într-un provider. Aceasta librărie ne permite să citim și să actualizăm date din Redux Store.

2.6 Node.js

Scalabilitatea, latența și viteza sunt indicatori cheie pentru performanța aplicațiilor web. Să păstrezi latența mică și viteza mare nu este ușor. Node.js este un runtime de JavaScript care are o latență scăzută și un randament ridicat prin adoptarea unei abordări „non-blocking” pentru a răspunde cererilor. Node.js oferă suport pentru programarea asincronă, iar modul în care este construit îi permite să gestioneze mii solicitări simultane în același timp.

2.7 Amazon Web Services(AWS)

Amazon Web Services sau pe scurt AWS este cea mai folosită platformă cloud din lume și furnizează platforme de cloud computing și API-uri având peste 200 de servicii. Cloud computing înseamnă un serviciu găzduit pe internet care poate fi accesat de oriunde folosind internetul. Frontend-ul aplicației se afla într-un S3 Bucket care este un cloud storage oferit de către AWS. De asemenea, folosim Amazon Elastic Container Registry(ECR) care folosește Amazon Simple Storage Service(S3) pentru stocare pentru a face imaginile containerului ușor accesibile și ne permite să găzduim noi containere. Amazon ECR transferă imaginile din container prin HTTPS și le criptează în mod automat. În cele din urmă, backendul este deployed într-un ECR de unde este preluat și folosit de un ECS, unde avem un serviciu care folosește Fargate pentru rularea dockerului, iar apoi există un load balancer și un entry de DNS în Amazon Route 53.

2.8 Azure AD

Azure Active Directory este un serviciu de management de acces care permite angajaților să acceseze resurse externe cum ar fi Microsoft 365, portalul Azure și mii alte aplicații SaaS. În acest mod angajatul se poate loga pe aplicația dezvoltată direct cu contul de Microsoft și nu este necesar să se mai înregistreze niciunde, iar procesul de onboarding se ușurează.

2.9 Microsoft Graph API

Microsoft Graph este un API public oferit de cei de la Microsoft care oferă access la datele serviciilor din Microsoft. Acest API este folosit pentru Single sign-on login care folosește OAuth 2.0 și returnează două token-uri unul folosit pentru a face request-uri HTTP la Graph Api, iar altul folosit pentru a face request-uri HTTP la server. De asemenea, API-ul mai este folosit în diverse locuri din aplicație de exemplu pentru a obține o lista cu toții angajații în cazul adminului etc.

2.10 Express.js

Express.js, sau simplu Express, este un framework folosit de către servere pentru a construi RESTful APIs în Node.js. Express.js este un strat deasupra de Node.js care ne ajuta să administrăm rutele. Cu ajutorul acestui framework sunt declarate rutele pentru comunicarea prin protocolul HTTP dintre client și server.

2.11 Mindee OCR

Mindee este o platforma care ajuta dezvoltatorii de aplicații web să automatizeze aplicațiile oferind API-uri de Optical Character Recognition pentru mai multe limbaje de programare. OCR este o tehnologie care recunoaște textul dintr-o imagine. Este cel mai des folosită pentru a recunoaște text în documente scanate și imagini. Mindee oferă un API construit special pentru a extrage date de pe bonurile fiscale care este antrenat doar cu bonuri fiscale, așadar nu este un algoritm general de recunoaștere de caractere ci este antrenat în mod special pentru a avea o performanță bună în cazul bonurilor fiscale de orice tip.

2.12 Sequelize

Sequelize este un ORM(Object Relațional Mapping) folosit în Node.js pentru a ușura lucrul cu bazele de date. Acesta suportă multiple dialecte printre care Oracle, Postgres, MySQL, MariaDB, SQLite și SQL Server etc. Sequelize are foarte multe funcționalități cum ar fi:

1. Suport pentru lucrul cu tranzacții,
2. Relații de one-to-one, one-to-many, many-to-one și many-to-many

3. Hook-uri pentru AfterCreate, AfterUpdate, BeforeCreate, BeforeUpdate și tot așa pentru fiecare operație asupra bazei de date, care iau locul triggerelor
4. Eager și lazy loading
5. Migrări pentru crearea tabelor și a vizualizărilor
6. Seedere pentru popularea tabelor și multe altele

2.13 MySQL

MySQL este un sistem de gestionare a bazelor de date relaționale. Numele său este o combinație dintre “My”, numele fiicei co-fondatorului Michael Widenius, și “SQL”, acronimul pentru Structured Query Language. Principalul motiv pentru care am ales această bază de date este că în MySQL există relații (many-to-many, one-to-many, one-to-one și many-to-one) pentru a conecta entități între ele și pentru că există operația de JOIN care ne permite să aducem date din mai multe tabele într-un singur query. Avantajele acestei baze de date sunt că datele sunt normalizate pentru a asigura integritatea lor, există relații între entități, datele sunt distribuite în mai multe tabele, scalează vertical însă există o limită deoarece un calculator are o limită a puterii de calcul, dar există o șansă mare să nu atingem această limită pentru că este vorba de foarte multă putere de calcul. În MySQL, tabelele sunt fixe adică nu putem avea o intrare cu 3 coloane și o intrare cu 5 coloane ca și în bazele de date NoSQL, dar asta reprezintă un dezavantaj doar în cazul în care vrem să fim flexibili. De asemenea, bazele de date MySQL sunt mai lente decât cele NoSQL însă acest lucru se simte atunci când avem mii de requesturi pe secundă și operăm cu milioane de intrări în baza de date, iar pentru o aplicație de deconturi care este folosită în firme, acesta nu este cazul.

2.14 Docker

Docker este o platformă care oferă servicii de virtualizare la nivelul sistemului de operare și ne ajută să lucrăm cu containere. Un container este o parte din sistemul de operare care are anumite limite cu privire la câte resurse poate să folosească. Containerele sunt construite cu ajutorul namespace-urilor care ne permit să împărțim sistemul de operare în mai multe sisteme de operare virtualizate și izolate. Aceste sisteme de operare virtuale conțin toate resursele sistemului de operare original și se numesc containere. Fiecare container are propriul sistem de fișiere, process tree, propriul root user și folosesc kernelul hostului. De asemenea, fiecare container are procesul rădăcină PID=1 și în acest mod un container nu are nici o idee că în sistemul de operare poate exista și alte containere. Grupurile de control ajută și ele la construirea unui container și au rolul de a stabili cât CPU, RAM și disk poate să folosească un container. Docker Engine este soft-ul care hostează containerele, iar acesta este format din:

1. Daemon care reprezintă un REST API la care se fac requesturi
2. ContainerD folosit pentru execuție și pentru ciclul de viață a unui container, folosit pentru a opri/porni un container

3. OCI care realizează interfață cu kernelul

2.15 ESLint

ESLint este un tool pentru analiza codului care identifica problemele care țin de formatarea de cod din JavaScript și TypeScript. Regulile din ESLint sunt configurabile și se pot defini reguli personalizabile în cazul în care este nevoie. ESLint se ocupă atât de calitatea codului cât și de stilul codului. Folosind ESLint codul este mult mai organizat și este formatat în același mod în toate fișierele rezultând un cod mai ușor de citit și ordonat.

2.16 Github Actions

GitHub Actions este o platformă de continuous integration și continuous delivery(CI/CD) care ne permite să automatizăm buildul, testele și pipeline-urile pentru deployment. Putem să cream workflow-uri care fac build și testează fiecare pull request din repository. GitHub furnizează mașini virtuale de Linux, Windows și macOS pentru a rula workflow-uri. Folosind GitHub Actions tot procesul de development devine automat deoarece se poate crea câte un workflow pentru fiecare branch (production, develop etc), iar în acest mod se poate deploy automat la noua versiune de cod permițând astfel CI/CD și ne mai fiind nevoie să facem deploy manual. Cu ajutorul acestei platforme putem configura un workflow care să fie declanșat atunci când un eveniment apare în repository, cum ar fi un pull request sau un task care a fost creat. Fiecare workflow conține unul sau mai multe joburi care pot fi să ruleze în paralel, fie să ruleze secvențial, acest lucru este configurat din fișierele cu extensia .yml care descriu fiecare workflow și toți pașii pe care acesta îi are. Fiecare job va rula înăuntrul propriei mașini virtuale sau înăuntrul unui container și are unul sau mai mulți pași care fie rulează un script, fie o acțiune pe care am definit-o.

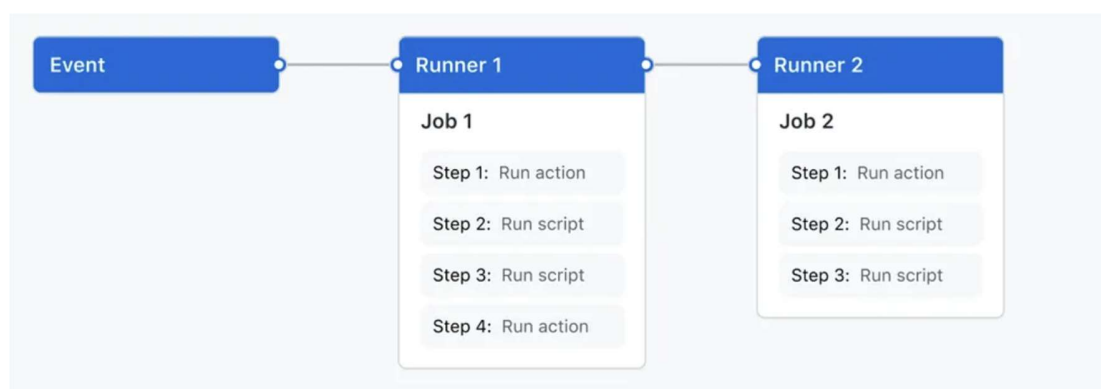


Figura 2.1 Exemplu Github Action

2.17 Terraform

Terraform este un tool care ne permite să descriem complet infrastructura unui proiect software prin cod. Deoarece serviciile folosite pot fi de la diferiți furnizori cum ar fi AWS, CloudFare, Heroku sau alții, Terraform te ajută să definești toată infrastructura în paralel pentru toți furnizorii. De asemenea, odată ce am definit întreaga infrastructura, nu mai trebuie să intrăm manual, de exemplu, pe AWS ca să creăm infrastructura de mână (de exemplu un ECS), putem rula terraform apply, iar apoi toată infrastructura este creată automat pentru noi.

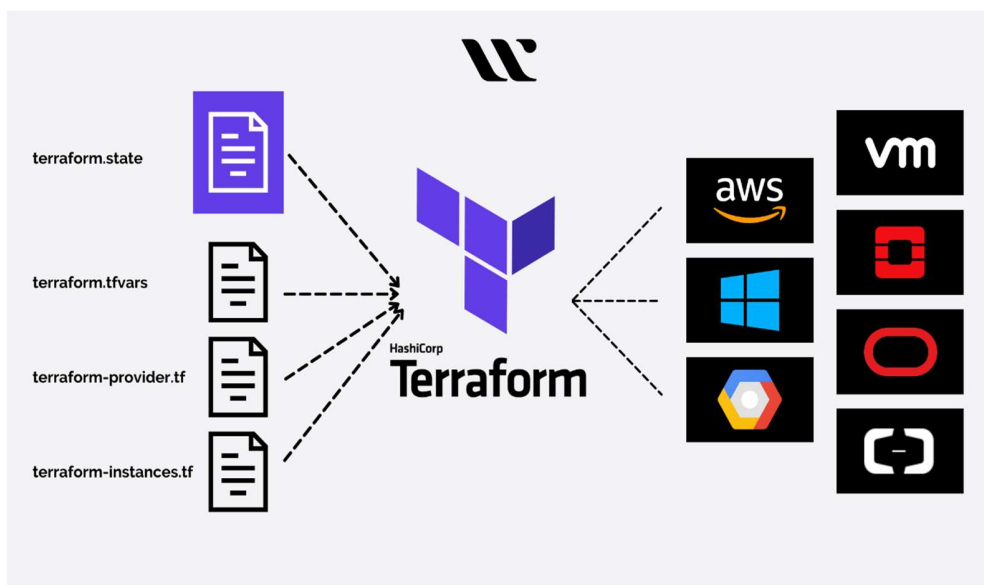


Figura 2.2 Ilustrare utilizare Terraform

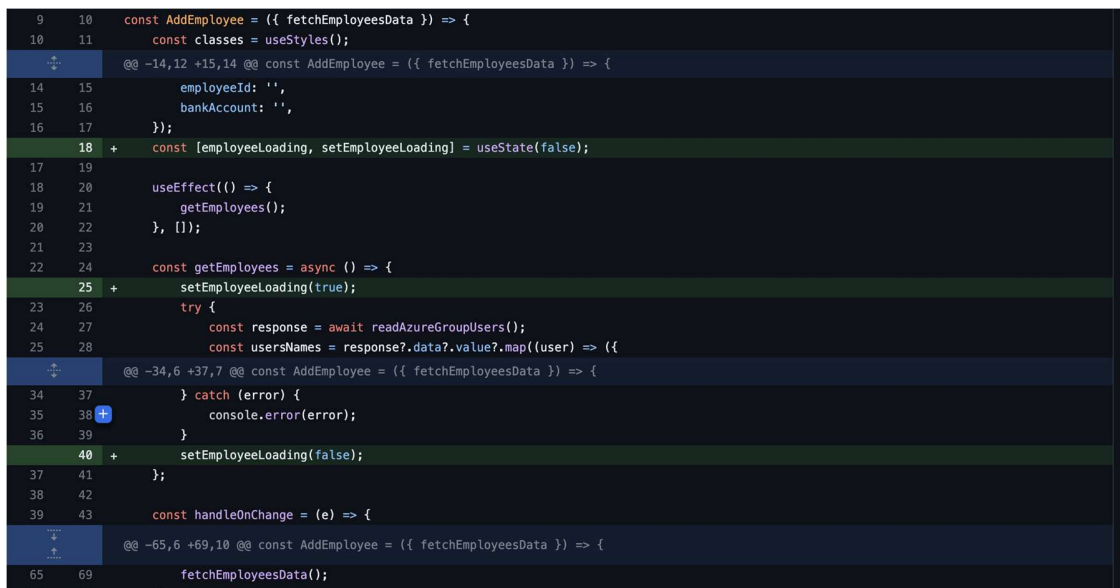
2.18 Versionarea Codului

Fiecare proiect software trebuie să folosească un sistem de versionare a codului pentru a putea reveni la versiuni anterioare în cazul unui greșeli și pentru ca proiectul să fie dezvoltat de mai multe persoane. Versionarea codului reprezintă înregistrarea tuturor versiunilor ale fișierelor datorate modificării acestora astfel încât se poate reveni la o versiune anterioară în cazul în care este nevoie. Avantajul versionării este acela că în cazul în care s-a făcut o modificare care a provocat că sistemul să nu mai meargă se poate reveni la o versiune anterioară care este funcțională.

Git este un sistem distribuit de control al versiunilor care urmărește schimbările din fiecare fișier și este folosit pentru coordonarea între programatorii care colaborează pe același proiect. Git are o licență software gratuită. De asemenea, printr-un fișier .gitignore îi putem comunica sistemului de control ce fișiere să nu urmărească în momentul versionării codului.

Github este un serviciu de hostare pentru software development care folosește sistemul de control al versiunilor Git. Pe GitHub se pot încarca toate branch-urile din proiect și se poate

urmării cu ușurință fiecare modificare adusa de fiecare commit. De asemenea, codul de pe un branch poate fi merge-uit cu codul de pe alt branch pentru unificarea functionalitatilor, iar diferențele dintre fișiere pot fi foarte ușor observate datorita interfetei pusa la dispoziție de către GitHub.



```
9 10 const AddEmployee = ({ fetchEmployeesData }) => {
10 11 const classes = useStyles();
@@ -14,12 +15,14 @@ const AddEmployee = ({ fetchEmployeesData }) => {
14 15 employeeId: '',
15 16 bankAccount: '',
16 17 });
18 + const [employeeLoading, setEmployeeLoading] = useState(false);
17 19
18 20 useEffect(() => {
19 21 getEmployees();
20 22 }, []);
21 23
22 24 const getEmployees = async () => {
25 + setEmployeeLoading(true);
23 26 try {
24 27 const response = await readAzureGroupUsers();
25 28 const usersNames = response?.data?.value?.map((user) => ({
@@ -34,6 +37,7 @@ const AddEmployee = ({ fetchEmployeesData }) => {
34 37 } catch (error) {
35 38 console.error(error);
36 39 }
40 + setEmployeeLoading(false);
37 41 });
38 42
39 43 const handleOnChange = (e) => {
@@ -65,6 +69,10 @@ const AddEmployee = ({ fetchEmployeesData }) => {
65 69 fetchEmployeesData();
```

Figura 2.3 Modificarile aduse unui fișier observate din interfața oferită de Github

2.19 Husky

Husky este un tool folosit pentru a rula comenzi sau scripturi înainte de a face commit sau înainte de a da push noii versiuni de cod folosind Git. Acest tool este foarte benefic deoarece putem seta, de exemplu, ca testele din proiect să ruleze înainte să dam push pe GitHub, iar în cazul în care un programator uită să ruleze testele înainte de git push, iar acestea eșuează, Husky nu vă permite acțiunea și vă prevenii astfel introducerea unui posibile erori în sistemul software.

3. Proiectare

3.1 Proiectarea Aplicației

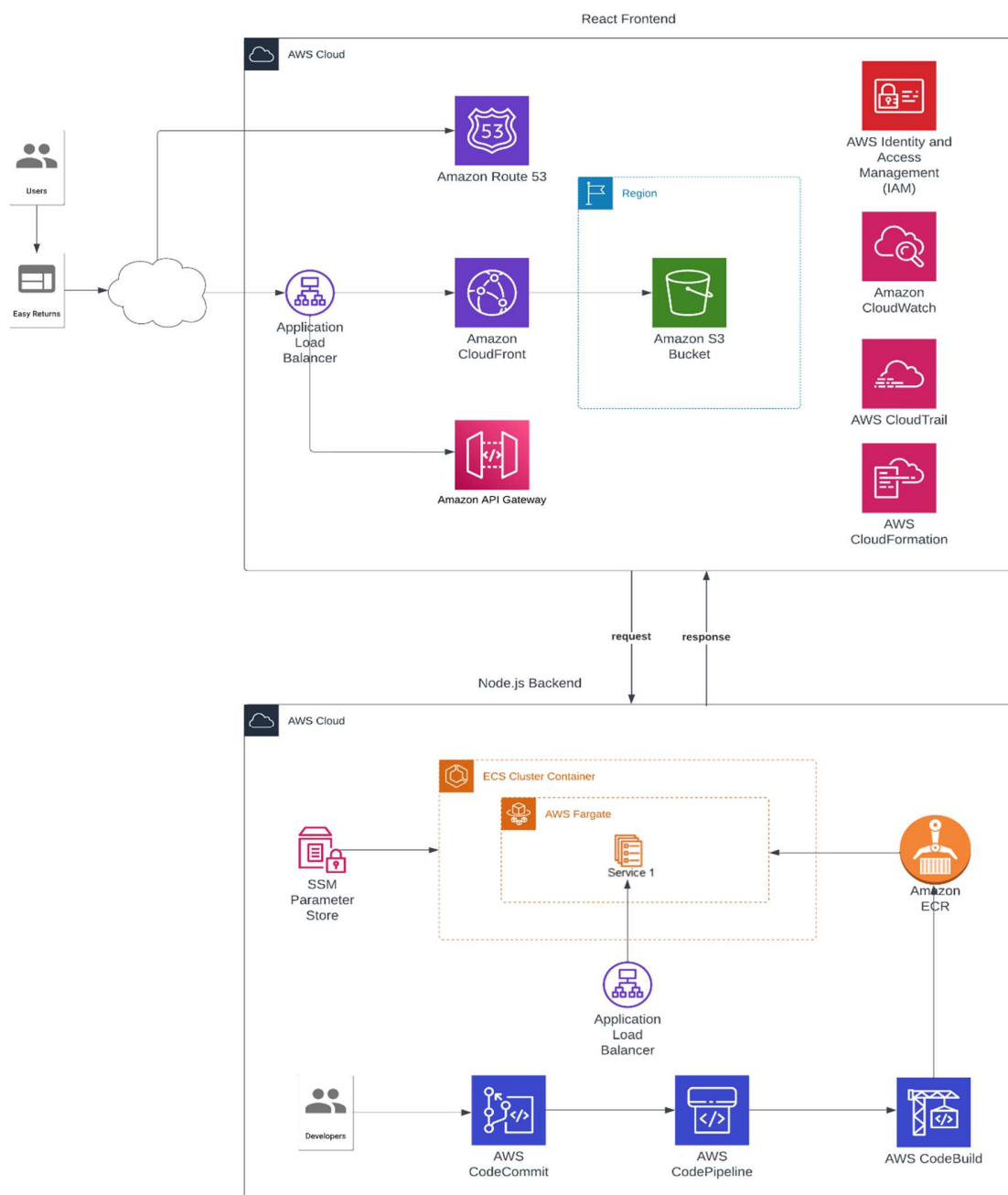


Figura 3.1 Infrastructura sistemului Easy Returns

În Figura 3.1 am realizat o diagramă care prezintă infrastructura aplicației. Observăm că partea de frontend a aplicației este deployed într-un Amazon S3 Bucket, iar că să fie secure și distribuit folosim Cloudfront. De asemenea, există un load balancer folosit pentru a distribui eficient traficul și câte o intrare de Amazon Route 53 pentru DNS(Domain Name System) și încă una pentru certificatul de ACM(AWS Certificate Manager) care are grijă de certificatele de SSL/TLS. Pe backend se face deploy în ECR(Elastic Container Registry) de unde este luat și folosit în ECS(Elastic Container Service), unde exista un serviciu care folosește Fargate pentru rularea dockerului. În final, la fel ca pe partea de backend, exista un load balancer și un entry de DNS în Amazon Route 53.

3.2 Diagrama UML a funcționalităților aplicației

Diagrama UML are rolul de a ilustra comportamentul sistemului în momentul în care un utilizator interacționează cu acesta, iar diagrama prezintă cele mai importante funcționalități din aplicație. În Figura 3.2 se observă că actorul principal este un User(administrator sau nu) care are rolul de a iniția, în mod direct, interacțiunea cu sistemul. Un utilizator se poate loga în aplicație folosind credențialele pentru SSO Login. Dacă încercarea de autentificare a avut succes, utilizatorul este redirectat pe prima pagină a aplicației care este reprezentată de pagina cu ordinele de deplasare. Din acest moment un utilizator normal poate interacționa cu sistemul în moduri variate.

Un utilizator normal are access la următoarele acțiuni: schimbarea pozei de profil, ieșirea din aplicație prin apăsarea butonului de logout, vizualizarea notificărilor prin care un utilizator normal este anunțat că un decont a fost acceptat/refuzat, vizualizarea propriilor ordine de deplasare, crearea unui nou ordin de deplasare, vizualizarea statusului în care se afla ordinul de deplasare creat, filtrarea și sortarea rezultatelor din lista de ordine de deplasare, navigarea pe pagina de deconturi, vizualizarea propriilor deconturi, crearea unui nou decont la care îi poate asocia mai multe bonuri fiscale, filtrarea și sortarea deconturilor, vizualizarea statusului unui decont, vizualizarea setărilor de unde își poate schimba/șterge poza de profil, vizualizarea permisiunilor și configurarea temei aplicației. De asemenea, exista și utilizatorul cu rol de administrator care are toate funcționalitățile unui utilizator normal descrise mai sus, însă acțiunile se aplica asupra tuturor angajaților în sensul că un administrator nu își vede doar propriile deconturi, ordine de deplasare ci vede pentru toată lumea, are filtre speciale pentru angajați etc. Funcționalitățile oferite în plus de rolul de administrator sunt: acceptarea/refuzarea/ștergerea de deconturi și ordine de deplasare(aceste acțiuni pot fi realizate și pentru mai multe intrări în același timp adică există operații de bulk delete, bulk approve, bulk reject), descărcarea unui fișier PDF care conține birocrația necesară ordinului de deplasare sau a decontului, exportarea unui fișier CSV în care se afla fiecare angajat și cu codul IBAN asociat acestuia, accesarea unei paginii exclusivă pentru admin în care acesta poate configura conturile bancare pentru fiecare angajat, iar în final, configurarea diurnei de baza pentru fiecare țară.

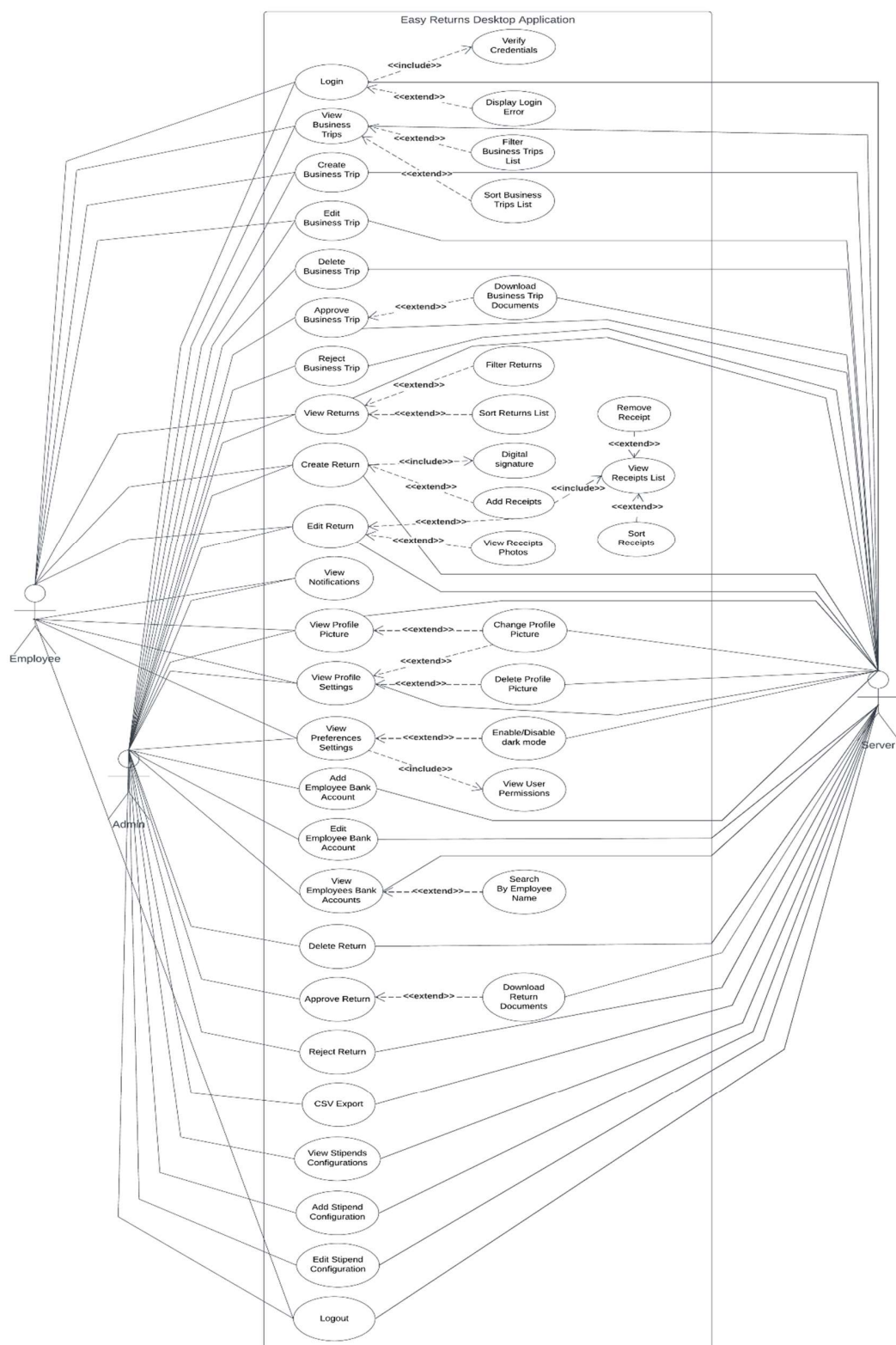


Figura 3.2 Diagrama UML

3.3 Navigarea in aplicatie

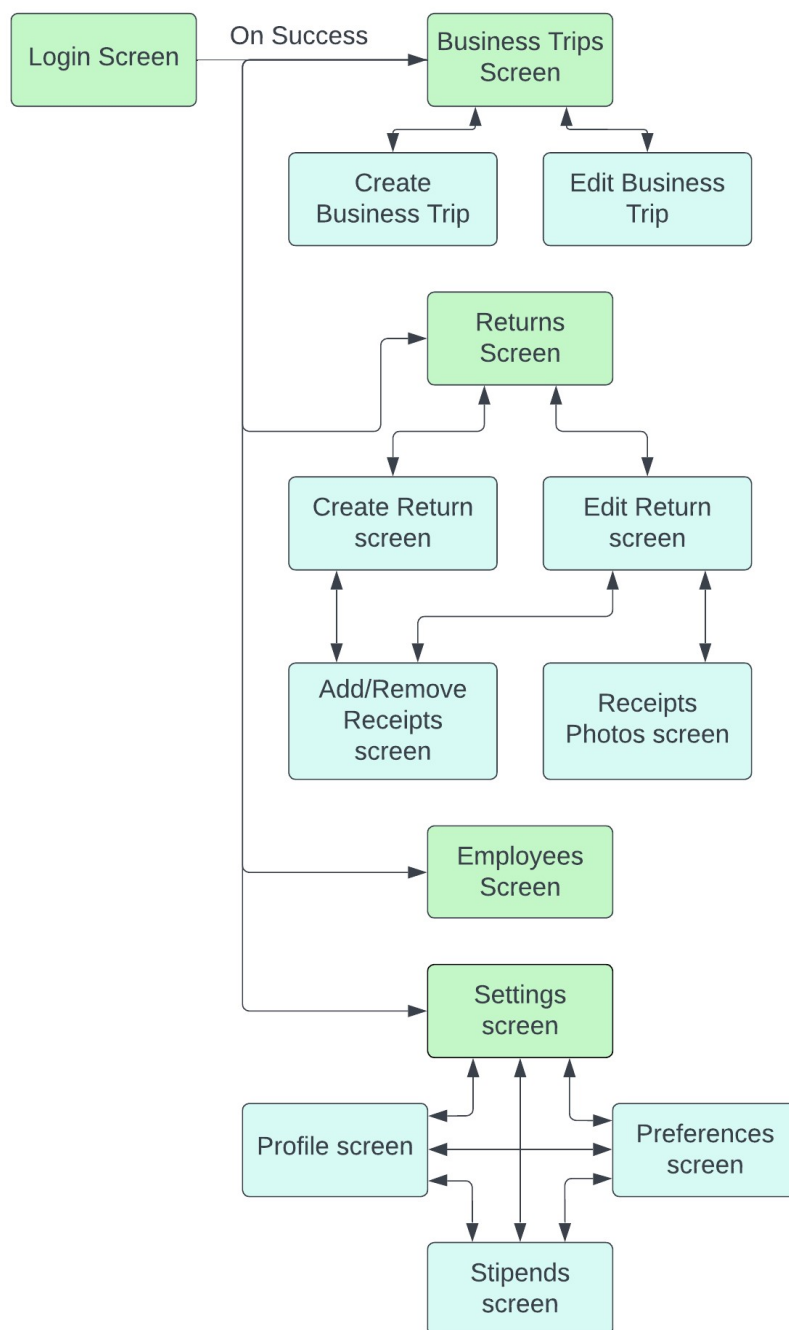


Figura 3.3 Navigarea în aplicație

Ecranele puse la dispoziție de către Easy Returns se pot observa în figura de mai jos. Se observă că primul pas pe care un angajat trebuie să îl facă că să acceseze aplicația este să se logheze folosind propriile credentiale. În caz contrar, în care un utilizator este logat, nici un ecran înafară de cel de login nu vă va fi disponibil.

După finalizarea pasului de autentificare, user-ul este redirectat către pagina de home care afișează propriile ordine de deplasare sau toate ordinele de deplasare în funcție de rolul utilizatorului. Pentru o navigare ușoară, bara de navigare (care este prezentă în Figura 3.4) este prezentă pe absolut toate paginile. Cele 5 paginii principale marcate în diagrama cu culoarea verde pot fi accesate de pe orice altă pagină din aplicație. De asemenea, dacă un utilizator este logat și încearcă să acceseze pagina de login, acesta va fi automat redirectat către home page adică lista cu ordinele de deplasare din Figura 3.4.

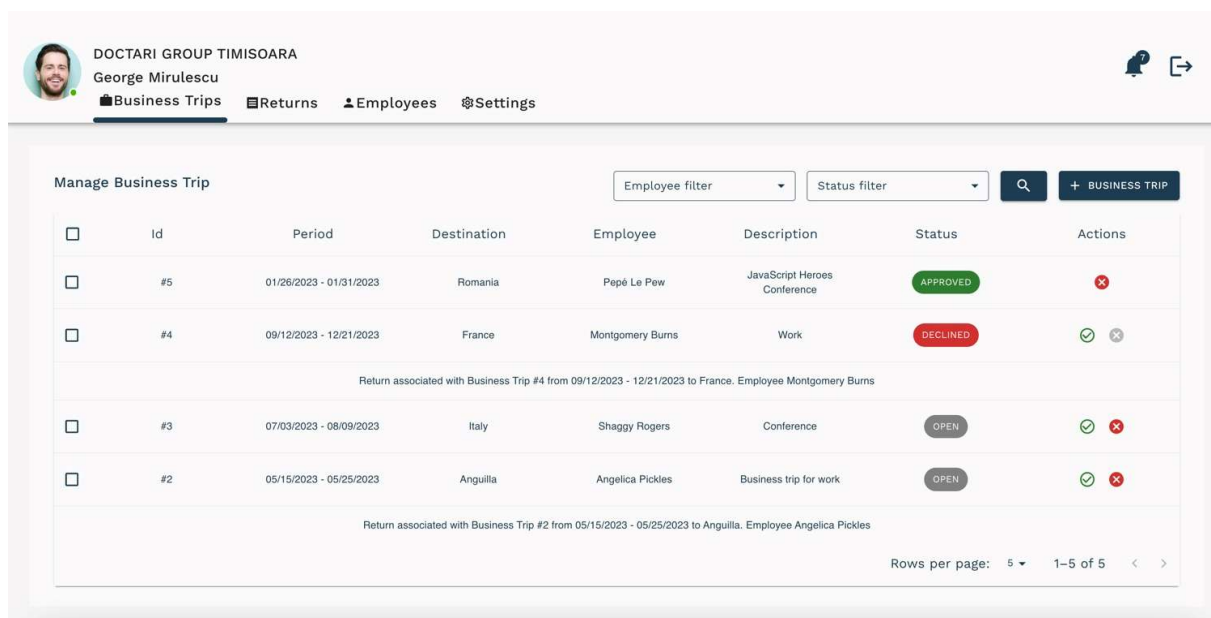


Figura 3.4 Pagina cu ordine de deplasare

3.4 Proiectarea bazei de date

Pentru stocarea resurselor am ales o bază de date relațională și anume MySQL. Bazele de date relaționale folosesc tabele. Toate intrările dintr-o anumită tabelă au aceleași coloane, deci nu oferă flexibilitatea bazelor de date nerelaționale. Pentru identificarea datelor am ales că id-ul să fie numeric și auto incrementat. Resursele folosite de către aplicație sunt organizate în modul următor:

1. **Tabela BankAccounts** este folosită pentru a salva informațiile cu privire la conturile bancare ale angajaților. Fiecare intrare din tabelă reprezintă un cont bancar care este asociat unui utilizator printr-o relație de one-to-one realizată prin cheia externă employeeId. Ceea ce este important de menționat este faptul că employeeId nu este un foreign key deoarece folosim SSO

Login cu oAuth2.0, iar utilizatorii sunt accesați prin Graph API, pus la dispoziție de Azure. Astfel, cum nu stocam utilizatorii în propria bază de date, nu se poate crea un foreign key. Tabela BankAccounts conține următoarele coloane:

- id INT PRIMARY KEY NOT NULL AUTO_INCREMENT
- employeeId VARCHAR(255) NOT NULL UNIQUE TRUE,
- employeeName VARCHAR(255) NOT NULL,
- bankAccount VARCHAR(255) NOT NULL,
- createdAt Datetime NOT NULL,
- updatedAt Datetime NOT NULL,
- deletedAt Datetime

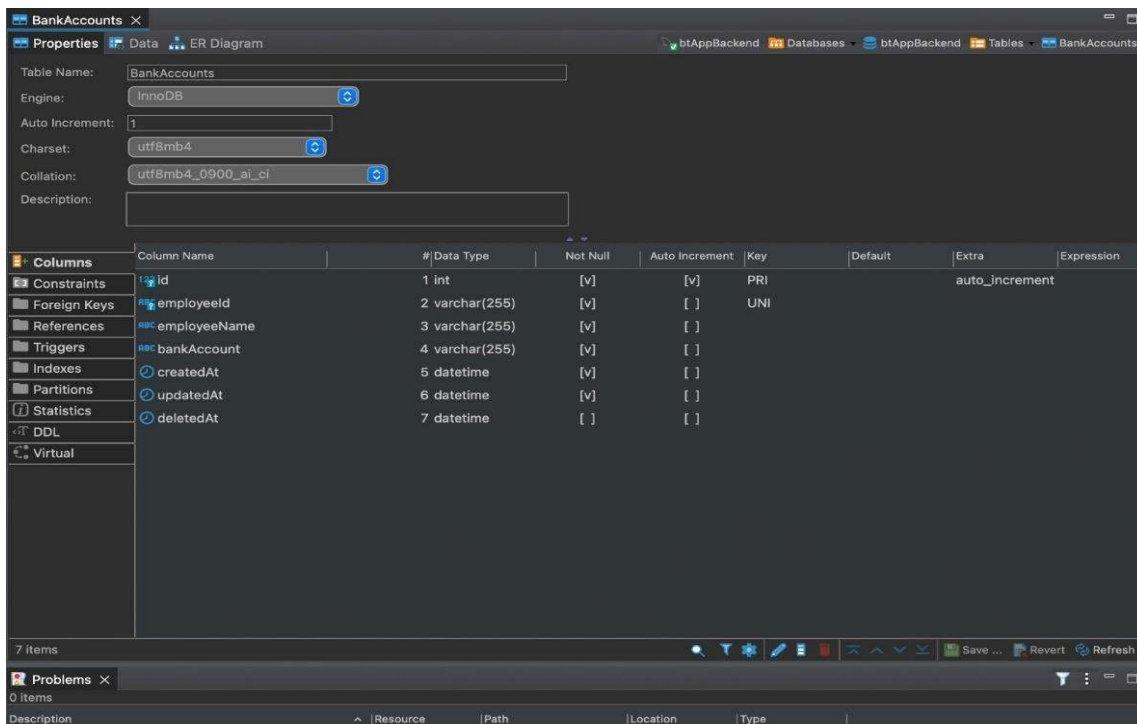


Figura 3.5 Tabela BankAccounts

2. **Tabela BusinessTrips** este folosită pentru a salva informațiile cu privire la ordinele de deplasare ale angajaților. Fiecare intrare din tabelă reprezintă un ordin de deplasare care are o relație de one-to-one cu un utilizator reprezentată prin coloana employeeId. Ceea ce este important de menționat este faptul că employeeId nu este un foreign key deoarece folosim SSO Login cu oAuth2.0, iar utilizatorii sunt accesați prin Graph API pus la dispoziție de Azure. Astfel, cum nu stocam utilizatorii în propria bază de date nu se poate crea un foreign key. Tabela BusinessTrips conține următoarele coloane:

- id INT PRIMARY KEY NOT NULL AUTO_INCREMENT
- status ENUM(,OPEN', ,APPROVED', ,DECLINED') NOT NULL DEFAULT ,OPEN'

- employee VARCHAR(255) NOT NULL
- employeeId VARCHAR(255) NOT NULL
- startDate Datetime NOT NULL
- endDate Datetime NOT NULL
- destination VARCHAR(255) NOT NULL
- stipend INT NOT NULL
- totalStipend INT NOT NULL
- description VARCHAR(255) NOT NULL
- signature BLOB,
- identityCard VARCHAR(255) NOT NULL
- adviserName VARCHAR(255)
- jobTitle VARCHAR(255) NOT NULL
- returnId INT FOREIGN KEY(returnId) REFERENCES Returns(id)
- createdAt Datetime NOT NULL,
- updatedAt Datetime NOT NULL,
- deletedAt Datetime

Column Name	#	Data Type	Not Null	Auto Increment	Key	Default	Extra	Expression
id	1	int	[v]	[v]	PRI		auto_increment	
status	2	enum('OPEN','AI')	[v]	[]		'OPEN'		
employee	3	varchar(255)	[v]	[]				
employeeId	4	varchar(255)	[v]	[]				
startDate	5	datetime	[v]	[]				
endDate	6	datetime	[v]	[]				
destination	7	varchar(255)	[v]	[]				
stipend	8	int	[v]	[]				
totalStipend	9	int	[v]	[]				
description	10	varchar(255)	[v]	[]				
signature	11	blob	[]	[]				
identityCard	12	varchar(255)	[v]	[]				
adviserName	13	varchar(255)	[]	[]				
jobTitle	14	varchar(255)	[v]	[]				
createdAt	15	datetime	[v]	[]				
updatedAt	16	datetime	[v]	[]				

Figura 3.6 Tabela BusinessTrips

3. **Tabela Notifications** este folosită pentru a salva informațiile cu privire la notificările pentru angajați. Fiecare intrare din tabelă reprezintă o notificare care are o relație de one-to-one cu un ordin de deplasare reprezentată prin coloana btId, o relație de one-to-one cu un decont reprezentată prin coloana returnId și o relație de one-to-one cu employeeId reprezentată de employeeId. Ceea ce este important de menționat este faptul că employeeId nu este un foreign key deoarece folosim SSO Login cu oAuth2.0, iar utilizatorii sunt accesați prin Graph API pus la dispoziție de Azure. Astfel, cum nu stocăm utilizatorii în propria bază de date nu se poate crea un foreign key. Tabela Notification conține următoarele coloane: id INT PRIMARY KEY NOT NULL AUTO_INCREMENT

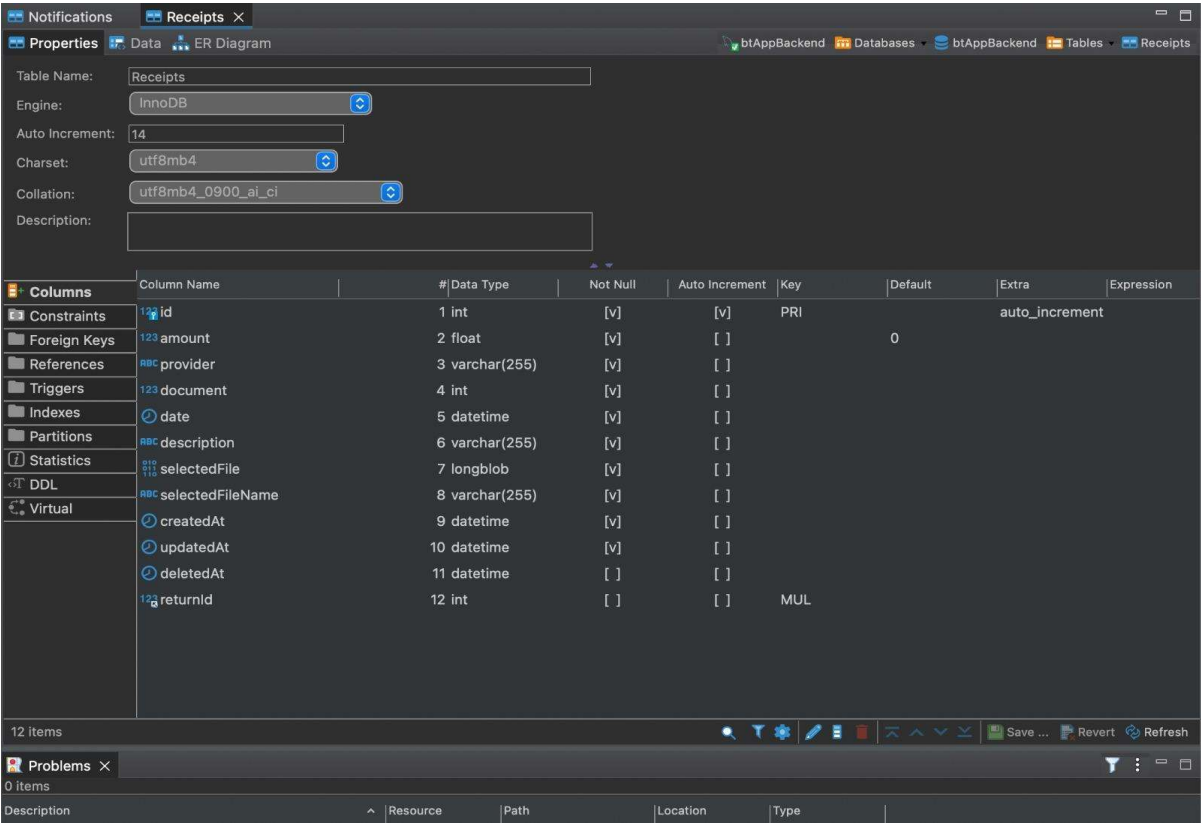
- text VARCHAR(255) NOT NULL
- returnId INT FOREIGN KEY(returnId) REFERENCES Returns(id)
- btId INT FOREIGN KEY(btId) REFERENCES BusinessTrips(id)
- employeeId VARCHAR(255) NOT NULL UNIQUE TRUE
- isSeen TINYINT(1) NOT NULL
- createdAt Datetime NOT NULL,
- updatedAt Datetime NOT NULL,
- deletedAt Datetime

Column Name	#	Data Type	Not Null	Auto Increment	Key	Default	Extra	Expression
id	1	int	[v]	[v]	PRI			auto_increment
text	2	varchar(255)	[v]	[]				
createdAt	3	datetime	[v]	[]				
updatedAt	4	datetime	[v]	[]				
deletedAt	5	datetime	[]	[]				
btId	6	int	[]	[]	MUL			
returnId	7	int	[]	[]	MUL			
employeeId	8	varchar(255)	[v]	[]				
isSeen	9	tinyint(1)	[v]	[]		0		

Figura 3.7 Tabela Notifications

4. **Tabela Receipts** este folosită pentru a salva informațiile cu privire la bonurile fiscale asociate deconturilor. Fiecare intrare din tabelă reprezintă un bon fiscal care are o relație de one-to-one cu un ordin de deplasare reprezentată prin coloana returnId din tabela BusinessTrips
 Tabela Notification conține următoarele coloane:

- id INT PRIMARY KEY NOT NULL AUTO_INCREMENT
- amount FLOAT NOT NULL DEFAULT 0
- provider VARCHAR(255) NOT NULL
- document INT NOT NULL
- date Datetime NOT NULL
- description VARCHAR(255) NOT NULL
- selectedFile LONGBLOB NOT NULL
- selectedFileName VARCHAR(255)
- returnId INT FOREIGN KEY(returnId) REFERENCES Returns(id)
- createdAt Datetime NOT NULL,
- updatedAt Datetime NOT NULL,
- deletedAt Datetime



Column Name	#	Data Type	Not Null	Auto Increment	Key	Default	Extra	Expression
id	1	int	[v]	[v]	PRI		auto_increment	
amount	2	float	[v]	[]		0		
provider	3	varchar(255)	[v]	[]				
document	4	int	[v]	[]				
date	5	datetime	[v]	[]				
description	6	varchar(255)	[v]	[]				
selectedFile	7	longblob	[v]	[]				
selectedFileName	8	varchar(255)	[v]	[]				
createdAt	9	datetime	[v]	[]				
updatedAt	10	datetime	[v]	[]				
deletedAt	11	datetime	[]	[]				
returnId	12	int	[]	[]	MUL			

Figura 3.8 Tabela Receipts

5. **Tabela Returns** este folosită pentru a salva informațiile cu privire la deconturi. Fiecare intrare din tabelă reprezintă un decont care are o relație de one-to-one cu un utilizator reprezentată prin coloana employeeId. Ceea ce este important de menționat este faptul că employeeId nu este un foreign key deoarece folosim SSO Login cu oAuth2.0, iar utilizatorii sunt accesați prin Graph API pus la dispoziție de Azure. Astfel, cum nu stocam utilizatorii în propria bază de date nu se poate crea un foreign key. Tabela Returns conține următoarele coloane: id INT PRIMARY KEY NOT NULL AUTO_INCREMENT

- status ENUM('OPEN', 'APPROVED', 'DECLINED') NOT NULL DEFAULT 'OPEN'
- amount FLOAT NOT NULL DEFAULT 0
- employee VARCHAR(255) NOT NULL
- employeeId VARCHAR(255) NOT NULL
- returnDate Datetime NOT NULL
- description VARCHAR(255) NOT NULL
- eSignature BLOB
- aSignature BLOB
- adviserName VARCHAR(255)
- documentNumber INT NOT NULL DEFAULT 0
- createdAt Datetime NOT NULL,
- updatedAt Datetime NOT NULL,
- deletedAt Datetime

Column Name	#	Data Type	Not Null	Auto Increment	Key	Default	Extra	Expression
id	1	int	[v]	[v]	PRI		auto_increment	
status	2	enum('OPEN','AI	[v]	[]		'OPEN'		
amount	3	float	[v]	[]				
employee	4	varchar(255)	[v]	[]				
employeeId	5	varchar(255)	[v]	[]				
returnDate	6	datetime	[v]	[]				
description	7	varchar(255)	[v]	[]				
eSignature	8	blob	[]	[]				
aSignature	9	blob	[]	[]				
adviserName	10	varchar(255)	[]	[]				
createdAt	11	datetime	[v]	[]				
updatedAt	12	datetime	[v]	[]				
deletedAt	13	datetime	[]	[]				
documentNumber	14	int	[v]	[]		0		

Figura 3.9 Tabela Returns

6. **Tabela Stipends** este folosită pentru a salva informațiile cu privire la valorile diurnelor pentru fiecare țară. Fiecare intrare din tabela reprezintă o diurna. Tabela Stipends conține următoarele coloane:

- id INT PRIMARY KEY NOT NULL AUTO_INCREMENT
- country VARCHAR(255) NOT NULL UNIQUE
- amount INT NOT NULL DEFAULT 0
- createdAt Datetime NOT NULL,
- updatedAt Datetime NOT NULL,
- deletedAt Datetime

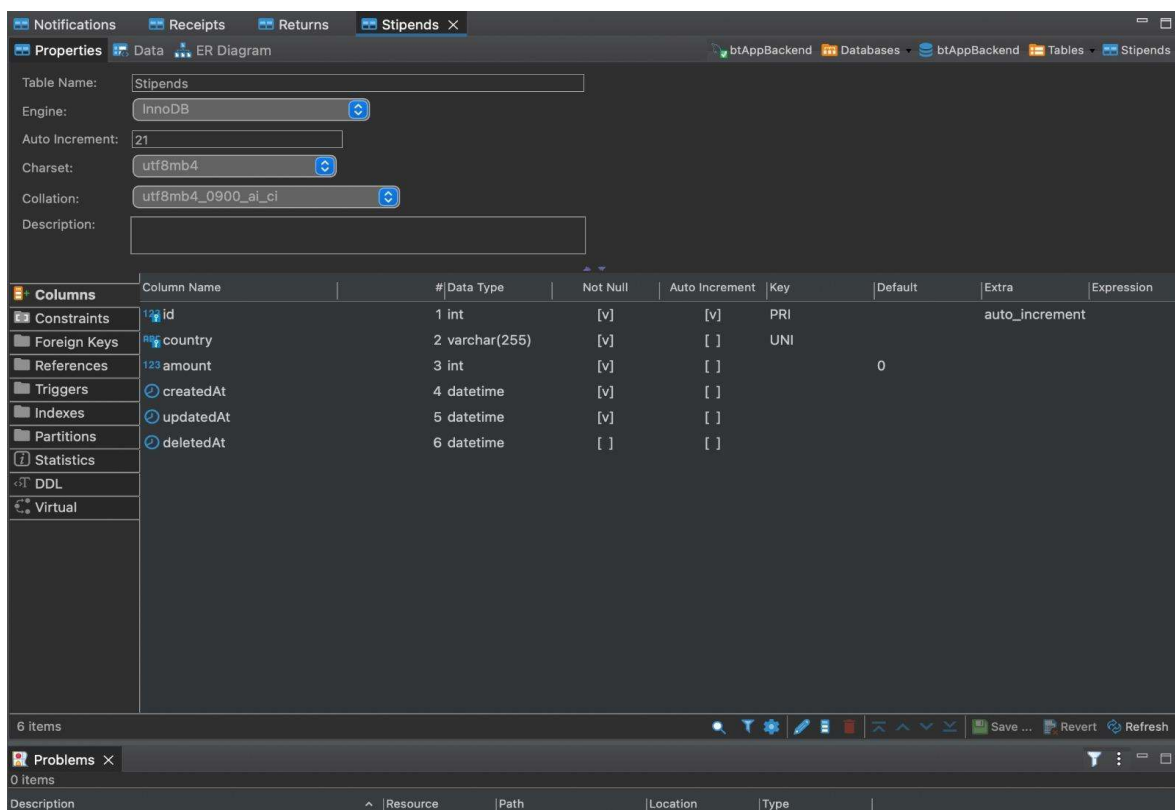
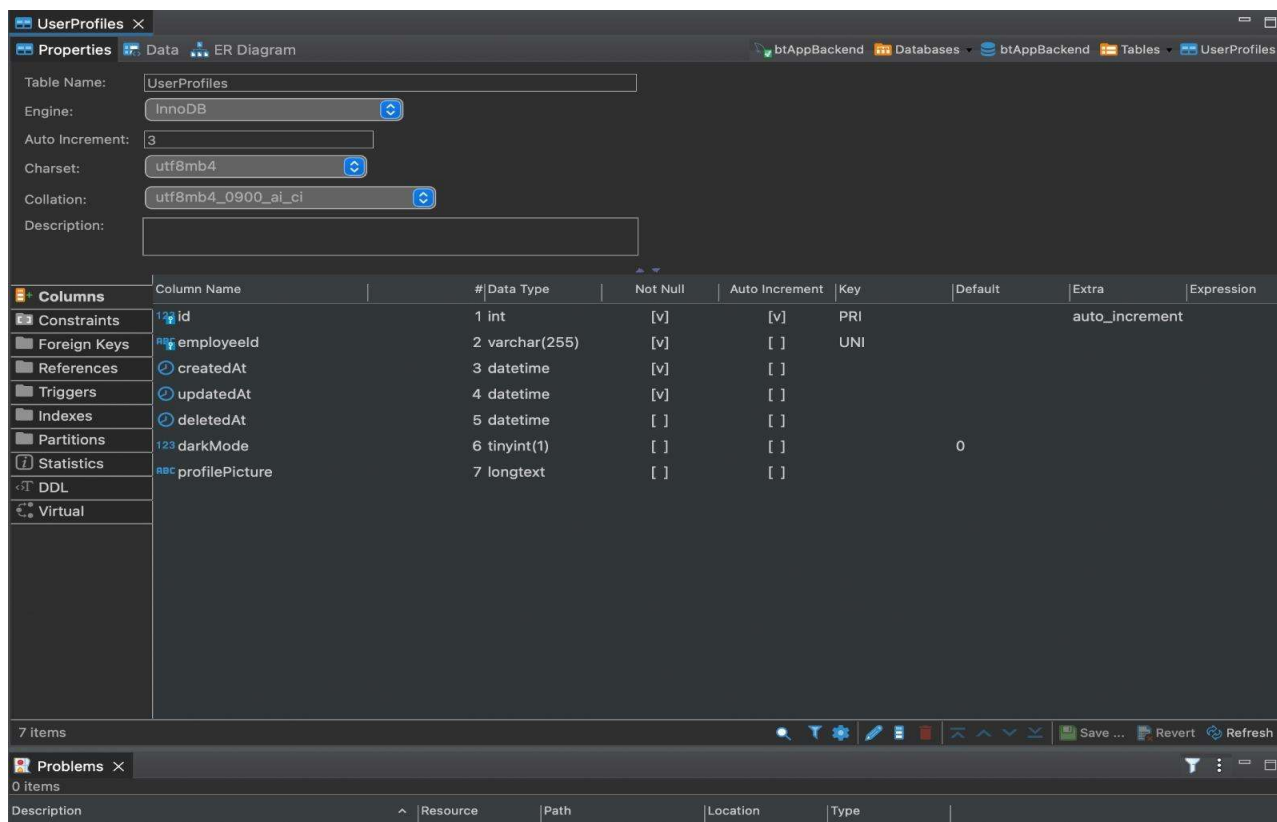


Figura 3.10 Tabela Stipends

7. **Tabela UserProfiles** este folosită pentru a salva informații legate de utilizatori. Fiecare intrare din tabelă reprezintă profilul unui user și are o relație de one-to-one cu un utilizator reprezentată prin coloana employeeId. Tabela UserProfiles conține următoarele coloane:

- id INT PRIMARY KEY NOT NULL AUTO_INCREMENT
- employeeId VARCHAR(255) NOT NULL UNIQUE
- darkMode TINYINT(1) DEFAULT 0
- profilePicture LONGTEXT



UserProfiles

Table Name: UserProfiles
 Engine: InnoDB
 Auto Increment: 3
 Charset: utf8mb4
 Collation: utf8mb4_0900_ai_ci

Column Name	#	Data Type	Not Null	Auto Increment	Key	Default	Extra	Expression
id	1	int	[v]	[v]	PRI		auto_increment	
employeeId	2	varchar(255)	[v]	[]	UNI			
createdAt	3	datetime	[v]	[]				
updatedAt	4	datetime	[v]	[]				
deletedAt	5	datetime	[]	[]				
darkMode	6	tinyint(1)	[]	[]		0		
profilePicture	7	longtext	[]	[]				

Figura 3.11 Tabela UserProfiles

4. Implementarea Aplicației

În această secțiune voi prezenta și explica în detaliu modul în care au fost implementate anumite funcționalități. Întrucât există foarte multe funcționalități le voi prezenta pe cele mai importante.

4.1 Implementarea autentificării folosind SSO Login cu OAuth2.0

Partea de autentificare a aplicației folosește Azure Active Directory single sign on care este o metodă de autentificare ce permite utilizatorilor să se logheze în aplicații folosind un singur set de credențiale pentru mai multe sisteme software independente. OAuth 2.0 este un protocol de autorizare care permite unui utilizator să aibă access la resursele sale. Acesta este construit să lucreze cu Hypertext Transfer Protocol(HTTP).

OAuth 2.0 funcționează în modul următor: clientul face un request că să acceseze resursele, primul răspuns va fi un cod care va fi folosit pentru următorul request care o să returneze un access token și un id token. Acces tokenul este folosit pentru a face requesturi la API-ul de la Microsoft, iar id tokenul conține informații despre utilizatorul autentificat în funcție de ceea ce s-a cerut în request, acesta poate conține numele, grupurile din care face parte user-ul etc. După ce am obținut cele doua tokenuri, se mai face un request la Azure care returnează un al treilea token, denumit, în aplicație, apiToken care este folosit pentru a face requesturi la backendul nostru implementat în Node.js.

Primul pas pentru implementarea autentificării este configurarea librăriei oferită de Microsoft, MSAL, pentru a seta un clientId care indica clientul la care are loc autentificarea, un redirectUri care reprezintă url-ul la care se face redirect în momentul în care autentificarea a avut success și un postLogoutRedirectUri care indică pagina pe care se face redirect în momentul în care un utilizator iese din cont. De asemenea, există și configurări pentru cache: cacheLocation care indică locul în care vă fi stocat cache-ul și storeAuthStateInCookie care este pe fals deoarece nu vrem să stocăm starea de autentificare în cookie. Atributul system este folosit pentru configurarea unui logger care ne ajută la debug în cazul în care este nevoie.

În cele din urma, trebuie să configurăm permisiunile pe care le cerem în momentul autentificării care influențează răspunsul din idToken care conține informații despre utilizator. De exemplu, dacă în aplicație avem un feature ce presupune schimbarea pozei de profil din azure a utilizatorului logat, avem nevoie de permisiunea „User.ReadWrite.All”, pentru a citi grupurile din care face parte avem nevoie de alta permisiune și tot așa. Permisunile necesare pentru fiecare acțiune pot fi găsite pe documentația oferită de graph API de la Microsoft. În Figura 4.1 observăm permisiunile necesare pentru a putea accesa poza de profil a unui utilizator.

To retrieve the profile photo of a user		
Permission type	Least privileged permissions	Higher privileged permissions
Delegated (work or school account)	User.Read	User.ReadBasic.All, User.Read.All, User.ReadWrite, User.ReadWrite.All
Delegated (personal Microsoft account)	User.Read	User.ReadWrite
Application	User.Read.All	User.ReadWrite.All

Figura 4.1 Permisuni necesare pentru a accesa poza de profil a unui utilizator

Codul pentru configurarea librăriei MSAL:

```

45 export const loginRequest = {
46   scopes: ['User.Read.All', 'Group.Read.All'],
47 };
48
49 export const protectedResources = {
50   scopes: [`api://${process.env.REACT_APP_API_ID}/access_as_user`],
51 };
52

```

Cod 4.1 Permisunile cerute la login si URL-ul la care se face requestul

```

3 export const msalConfig = {
4   auth: {
5     clientId: process.env.REACT_APP_CLIENT_ID,
6     authority: `https://login.microsoftonline.com/${process.env.REACT_APP_TENANT_ID}/`,
7     redirectUri: process.env.REACT_APP_URI,
8     postLogoutRedirectUri: process.env.REACT_APP_URI + '/login',
9   },
10  cache: {
11    cacheLocation: 'sessionStorage', // This configures where your cache will be stored, it can be sessionStorage
12    storeAuthStateInCookie: false, // Set this to "true" if you are having issues on IE11 or Edge
13  },
14  system: {
15    loggerOptions: {
16      loggerCallback: (level, message, containsPii) => {
17        if (containsPii) {
18          return;
19        }
20        switch (level) {
21          case msal.LogLevel.Error:
22            console.error(message);
23            return;
24          case msal.LogLevel.Info:
25            console.info(message);
26            return;
27          case msal.LogLevel.Verbose:
28            console.debug(message);
29            return;
30          case msal.LogLevel.Warning:
31            console.warn(message);
32            return;
33        }
34      },
35    },
36  },
37 };
38

```

Cod 4.2 Configurare MSAL

După ce am configurat librăria, următorul pas este implementarea logicii de autentificare. Pentru asta ne creem un obiect de tipul MSAL:

```
8      const myMSALObj = new msal.PublicClientApplication(msalConfig);
9
```

Cod 4.3 Construire obiect MSAL

Cu ajutorul căruia ne putem loga în aplicația folosind metoda „loginRedirect” care primește ca și parametru drepturile pentru care cerem permisiune. Drepturile pentru fiecare utilizator pot fi configurate din Azure Ad, iar dacă de exemplu noi cerem permisiune la un drept numite „Content.ReadAll” însă în Azure Ad userul respectiv nu are aceasta permisiune, autentificarea va eșua.

```
54  export function signIn() {
55      return myMSALObj.loginRedirect(loginRequest);
56  }
57
```

Cod 4.4 Funcția signIn

După ce ne-am autentificat cu success, în home page care este pagina cu ordinele de deplasare, se apelează automat funcția getBackendToken care face un request la graph API pentru a face rost de apiToken folosit pentru a face requesturi la backend-ul nostru:

```
24  export const getBackendToken = () => {
25      return getTokenRedirect(protectedResources);
26  };
```

Cod 4.5 Funcția getBackendToken

```
77  function getTokenRedirect(request) {
78      request.account = myMSALObj.getAccountByUsername(username);
79
80      return myMSALObj.acquireTokenSilent(request).catch((error) => {
81          console.warn( message: 'silent token acquisition fails. acquiring token using redirect');
82          if (error instanceof msal.InteractionRequiredAuthError) {
83              // fallback to interaction when silent call fails
84              return myMSALObj.acquireTokenRedirect(request);
85          } else {
86              console.warn(error);
87          }
88      });
89  }
90
```

Cod 4.6 Funcția getTokenRedirect

Într-un final, toți pașii de autentificare au fost parcuși, iar acum suntem logați în aplicație și avem un accessToken folosit pentru a face requesturi la microsoft graph API, un apiToken folosit pentru a face requesturi la backend-ul nostru și un idToken care conține informații despre utilizatorul logat.

În momentul în care se face un request la backend, apiToken-ul este verificat folosind o librărie numită Passport pe care am configurat-o să verifice tokenul emis de către Azure.

```
14  const passport = require('passport');
15  const BearerStrategy = require('passport-azure-ad').BearerStrategy;

42  app.use(passport.initialize());
43  passport.use(bearerStrategy);
```

Cod 4.7 Configurare librărie Passport

Iar apoi putem adăuga pentru fiecare ruta care vrem să fie secure un middleware care vă verifica automat tokenul:

```
81  app.use('/bt', passport.authenticate( strategy: 'oauth-bearer', options: {session: false}), btRoutes);
82  app.use('/return', passport.authenticate( strategy: 'oauth-bearer', options: {session: false}), returnRoutes);
83  app.use('/receipt', passport.authenticate( strategy: 'oauth-bearer', options: {session: false}), receiptRoutes);
84  app.use('/stipend', passport.authenticate( strategy: 'oauth-bearer', options: {session: false}), stipendRoutes);
85  app.use('/bankAccount', passport.authenticate( strategy: 'oauth-bearer', options: {session: false}), bankAccountRoutes);
86  app.use('/user/profile', passport.authenticate( strategy: 'oauth-bearer', options: {session: false}), userProfileRoutes);
87  app.use('/notifications', passport.authenticate( strategy: 'oauth-bearer', options: {session: false}), notificationsRoutes);
88  app.use('/emails', passport.authenticate( strategy: 'oauth-bearer', options: {session: false}), emailRoutes);
```

Cod 4.8 Rutele oferite de către server

Este important de precizat că tokenul este format din 3 părți:

1. Headerul care de obicei conține tipul tokenului, în cazul nostru JWT(Json Web Token) și algoritmul folosit pentru crearea tokenului.
2. Payloadul care conține datele despre o entitate, în cazul nostru utilizatorul, și meta data.
3. Semnătura care este folosită pentru a verifica integritatea tokenului.

În background, librăria passport verifică integritatea tokenului în felul următor: acesta reface semnătura tokenului folosindu-se de algoritmul indicat în header și de payload. Semnătura este mai apoi comparată cu semnătura cu care s-a făcut requestul. În cazul în care cele două semnături nu sunt identice, înseamnă că datele au fost modificate, iar tokenul este invalid, în cazul în care semnăturile sunt identice, tokenul este bun, și requestul poate continua.

4.2 Implementarea funcționalității de scanare a unui bon fiscal folosind un algoritm de OCR(Optical character recognition)

În momentul în care un user crează un decont, acesta poate crea mai multe bonuri fiscale care vor fi asociate cu decontul respectiv. Pentru a adăuga un bon fiscal există un form care fie poate fi completat de mână cu datele bonului, fie se poate selecta o poză de pe propriul calculator care este trimisă la backend unde are loc procesarea imaginii și extragerea datelor din imagine folosind un algoritm de OCR, răspunsul fiind automat completat în form-ul de pe UI fără a mai fi nevoie că un utilizator să introducă datele manual. Pe partea de UI, există un button „Scan Receipt” care ne permite să selectăm poza pe care dorim să o scanăm. Implementarea interfeței arată în felul următor:

```
<Button color='primary' variant='contained' onClick={choosePhotoToScan}>
  <AddIcon />
  Scan Receipt
  <TextField
    inputRef={scanReceiptRef}
    onChange={(e :... ) => scanReceipt(e)}
    className={classes.fileInput}
    required
    inputProps={{ accept: 'image/jpg, image/png' }}
    type='file'
    encType='multipart/form-data'
  />
</Button>
```

Cod 4.9 Implementarea interfeței pentru scanarea unui bon fiscal

Din Cod 4.9 observăm că există un buton care vă cheamă funcția „choosePhotoToScan” în momentul în care este apăsat. Deoarece am vrut să existe un buton care îți permite să atașezi o poză pentru că are un aspect mai frumos decât un input de tip file, butonul mai conține și un textField care este ascuns în pagina datorită clasei de CSS aplicate. Elementul text field are o referință către el precizată prin atributul „inputRef={scanReceiptRef}” care ne ajută să apăsăm pe input-ul invizibil în momentul în care apăsăm pe buton, că să se deschidă un pop-up ce ne permite să încărcăm o poză. Funcția choosePhotoToScan arată în felul următor:

```
const choosePhotoToScan = () => {
  scanReceiptRef.current.click();
};
```

Cod 4.10 Funcția choosePhotoToScan

Referința care se află pe elementul text field este declarată astfel:

```
const scanReceiptRef = useRef( initialValue: null);
```

Cod 4.11 Declararea unei referințe

Așadar, funcția choosePhotoToScan o să apese pe elementul text field, invizibil în pagină care vă deschide un input de tip fișier. Atributul type specifică faptul că input-ul acceptă doar fișiere, iar atributul accept din inputProps este folosit pentru a preciza singurele tipuri de fișiere care sunt permise. După ce utilizatorul a ales o poza, se va apela funcția de pe onChange, numită scanReceipt.

```
const scanReceipt = async (e) => {
  const receiptPhoto = e.target?.files[0];
  const base64 = await convertBase64(receiptPhoto);
  try {
    setLoading( value: true);
    const { data } = await post( obj: { image: base64.split(',')[1] }, endpoint: '/receipt/scan');

    setFormData((prevState) => ({
      ...prevState,
      date: data?.date,
      provider: data?.provider,
      amount: data?.totalAmount,
      selectedFile: base64,
      selectedFileName: e.target.files[0].name,
    }));

    setLoading( value: false);
  } catch (e) {
    console.error(e);
    setLoading( value: false);
  }
};
```

Cod 4.12 Funcția scanReceipt

Funcția scanReceipt convertește imaginea încărcată într-un string în baza 64, apoi se face un request HTTP de tip POST la ruta /receipt/scan și se trimite imaginea encodată în baza 64 în corpul requestului. De asemenea, componenta conține un state de tip boolean denumit loading, iar acesta este setat pe true, respectiv pe false la finalul logicii deoarece până când scanarea bonului se finalizează arătăm un alt ecran de loading pentru interacțiunea cu utilizatorul. Există și un state numit formData care conține valorile câmpurilor formularului de pe interfață. În acest mod, când setăm formData, formularul văzut de user va fi completat automat cu valorile primite ca și răspuns de la server. Așa cum am precizat mai sus, ruta care realizează scanarea bonului fiscal este /receipt/scan și folosește următoarea funcție:

```
const scanReceipt = async (req, res) => {
  const mindeeClient = new mindee.Client( {apiKey, throwOnError, debug}: { apiKey: process.env.MINDEE_API_KEY });

  const doc = mindeeClient.docFromBase64(req.body.image, { filename: 'myfile.png' });
  const respPromise = doc.parse(mindee.ReceiptV4);

  respPromise
    .then((resp : Response<ReceiptV4> ) => {
      if (resp.document === undefined) return;

      const response = {
        totalAmount: resp.document.totalAmount.value,
        provider: resp.document.supplier.value,
        date: resp.document.date.value
      };

      return res.status(200).send(response);
    })
    .catch((err) => {
      return res.status(500).send({
        error: {
          message: err.message
        }
      });
    });
};
```

Cod 4.13 Logica de extragere OCR de pe backend

Funcția `scanReceipt` de pe backend se conectează la client-ul Mindee printr-un API key preluat din variabilele de environment, convertește imaginea din corpul requestului din baza 64 în document folosind metoda `docFromBase64`, trimite documentul la un endpoint care realizează extragerea datelor de pe imaginea cu bonul fiscal și analizează predicțiile făcute de algoritmul de inteligență artificială. În momentul în care datele au fost extrase cu succes, se va intra pe callback-ul metodei „then” unde se formează un nou obiect în care stocăm `totalAmount` care reprezintă suma totală de pe bonul fiscal, `provider` care reprezintă furnizorul respectiv firma care a emis bonul fiscal și date care indică data în care a avut loc plata. În caz contrar, dacă request-ul eșuează se va intra pe callback-ul metodei „catch” care va trimite clientului nostru mesajul de eroare cu un code 500 care înseamnă „Internal Server Error”.

Răspunsul din partea serverului care realizează extragerea datelor conține mai multe atribute cum ar fi „filename” care este numele imaginii, total amount care este suma totală de pe bon, total net care este totalul fără TVA, timpul, procentul de TVA și multe altele. Din toate aceste câmpuri folosim doar de cele 3 (total amount, date și supplier name) de care avem nevoie.

```
----- Receipt V4 -----  
Filename: ffc127d-sample_receipt.jpg  
Total amount: 10.2  
Total net: 8.5  
Tip:  
Date: 2016-02-26  
Category: food  
Subcategory: restaurant  
Document type: EXPENSE RECEIPT  
Time: 15:20  
Supplier name: CLACHAN  
Taxes: 1.7 20.0% VAT  
Total taxes: 1.7  
Locale: en-GB; en; GB; GBP;  
-----
```

Figura 4.2 Raspunsul Algoritmului de OCR

4.3 Implementarea și folosirea containerelor de Docker

Toate containerele de pe partea de backend sunt definite în fișierul docker-compose.up din rădăcină proiectului. Avantajul major pentru care am folosit Compose este că îmi pot defini toate containerele într-un singur fișier care se afla în directorul rădăcină a proiectului, iar dacă cineva o să dorească să contribuie pe viitor, se poate conecta la baza de date folosită de server printr-o simplă comandă: docker-compose up. În acest mod, un programator nu mai trebuie să stea să își instaleze MySQL pe calculator, ci folosește o imagine de docker care instalează un container cu MySQL. Un container este o parte din sistemul de operare care are anumite limite cu privire la câte resurse poate să folosească. Că să construim un container se folosesc grupuri de control și namespaceuri.

Namespace-urile permit împărțirea sistemului de operare în mai multe sisteme de operare virtualizate și izolate care conțin toate resursele sistemului de operare original.

Grupurile de control stabilesc cât CPU, RAM și disk folosește un container. Fiecare container are propriul sistem de fișiere, arbore de procesare, propriul root user și folosește kernelul hostului.

Docker engine este format din:

1. daemon: reprezintă un REST API la care se fac requesturi
2. containerD: folosit pentru execuția și ciclul de viață a containerului, folosit pentru a porni/opri un container
3. OCI: interfață cu kernelul

Din CLI(command line interface) putem folosi comenzi de docker care fac requesturi la REST API-ul pus la dispoziție de daemon. Deoarece REST API-ul este decuplat de containere, în

producție putem da restart la daemon fără să dam restart și la containere, ceea ce este foarte important în producție deoarece nu vrem să facem aplicația indisponibilă în timpul restartului.

```
2     version: '3.1'
3
4     services:
5       adminer:
6         image: adminer
7         restart: always
8         ports:
9           - 9000:8080
10
11      db:
12        image: mysql
13        command: --default-authentication-plugin=mysql_native_password
14        restart: always
15        environment:
16          MYSQL_ROOT_PASSWORD: ${MYSQL_ROOT_PASSWORD}
17        ports:
18          - 3306:3306
19
```

Cod 4.14 definirea serviciilor din fișierul docker-compose.yml

Fișierul din Cod 4.14 definește două servicii: adminer și db. Serviciul adminer folosește o imagine care se numește “adminer”, iar portul 9000, la care poate fi accesat adminerul este mapat la portul 8080. Adminerul este o interfață care ne permite să interacționăm cu baza de date definită mai jos, în acest mod nu trebuie să ne chinăm din consolă și să intrăm în container folosind comanda “docker exec”. Serviciul db folosește imaginea de mysql care reprezintă baza de date de tipul MySQL. Am definit și o variabilă de mediu MYSQL_ROOT_PASSWORD care își ia valoarea din fișierul .env din rădăcină proiectului. Folosind comanda docker-compose-up containerele se formează automat pentru noi și acestea pot fi văzute în interfață de la docker.

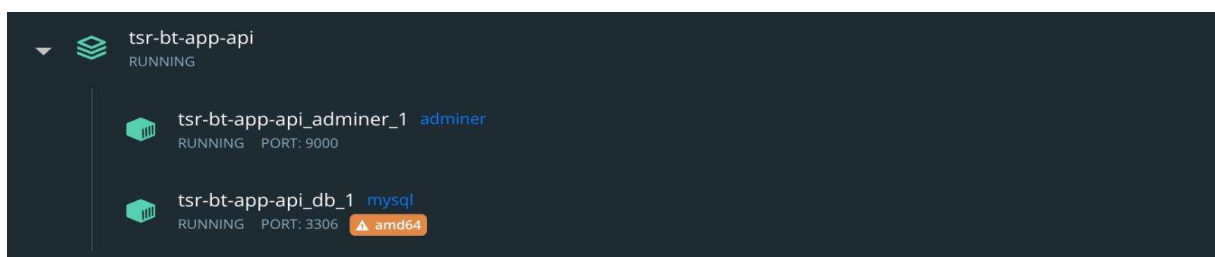


Figura 4.3 Containerele rezultate în urma comenzii docker-compose up

```
george@Mirulescu-MacBook-Pro ~ % docker ps
CONTAINER ID   IMAGE     COMMAND                  CREATED    STATUS    PORTS                               NAMES
4dd293767982   adminer   "entrypoint.sh docke..." 2 months ago Up 2 minutes    0.0.0.0:9000->8080/tcp    tsr-bt-app-api_adminer_1
4e9079ef79c6   mysql    "docker-entrypoint.s..." 2 months ago Up 2 minutes    0.0.0.0:3306->3306/tcp, 33060/tcp    tsr-bt-app-api_db_1
```

Figura 4.4 Containerele observate în terminal folosind docker ps

Containerele pot fi șterse folosind comanda “docker-compose down”. Până acum am precizat că folosim doua imagini numite mysql și adminer, dar ce sunt imaginile de docker defapt?

O imagine conține fișiere ale sistemului de operare și un manifest care este un fișier de tip JSON ce descrie cum funcționează totul împreună. O imagine este formată din mai multe straturi sau layere, iar modul în care sunt conectate este descris în fișierul de tip JSON. Straturile dintr-o imagine nu știu unele de altele, nu au referințe între ele. În final, ajungem la o concluzie: un container este o imagine care rulează, iar o imagine este practic un container oprit.

În momentul în care rulăm docker-compose up, imaginile și straturile sunt descărcate dintr-un repository în cazul în care nu le avem.

Cum se validează straturile unei imagini atunci când se descarcă o imagine de docker: fiecare strat are un hash(obținut printr-un algoritm criptografic) și astfel în fișierul JSON se află hash-urile fiecărui strat care se compară cu hash-urile straturilor descărcate și astfel are loc validarea. În concluzie, o imagine de docker este formată din mai multe fișiere(straturi) care se suprapun, iar modul de conectare al acestora este descris în fișierul JSON.

În cazul în care nu ar exista adminer-ul baza de date din interiorul containerului ar putea fi accesa parcurgând următorii pași:

1. Se rulează comanda “docker ps” ca să obținem numele containerului ce conține baza de date specificat în coloana NAMES.

```
george@Mirulescu-MacBook-Pro ~ % docker ps
CONTAINER ID   IMAGE      COMMAND                  CREATED        STATUS        PORTS                               NAMES
4dd293767982   adminer    "entrypoint.sh docke..." 2 months ago  Up 2 minutes  0.0.0.0:9000->8080/tcp             tsr-bt-app-api_adminer_1
4e9079ef79c6   mysql     "docker-entrypoint.s..." 2 months ago  Up 2 minutes  0.0.0.0:3306->3306/tcp, 33060/tcp  tsr-bt-app-api_db_1
```

Figura 4.5 Comanda docker ps

2. Intrăm în container folosind comanda “docker exec -it NUME_CONTAINER sh”

```
george@Mirulescu-MacBook-Pro ~ % docker exec -it tsr-bt-app-api_db_1 sh
#
```

Figura 4.6 Docker exec pentru a intra în container

sh – indică că vrem să deschidem un shellscript

it – ca să nu rulăm comanda în background și să putem executa alte comenzi în container

3. Ne conectăm la baza de date:

```
# mysql -u root -p
Enter password:
```

Figura 4.7 Conectarea la baza de date folosind linia de comandă

Opțiunea -u ne permite să scriem user-ul cu care vrem să ne conectăm, iar opțiunea -p este folosită pentru a introduce parola necesară pentru a ne conecta la MySQL. Parola este cea specificată în fișierul docker-compose.yml și preluată din variabilele de environment. După ce am introdus parola și conectarea a avut loc cu succes putem folosi comenzi mysql pentru a vizualiza bazele de date și tabelele:

```
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 8
Server version: 8.0.28 MySQL Community Server - GPL

Copyright (c) 2000, 2022, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> show databases;
+-----+
| Database |
+-----+
| btAppBackend |
| information_schema |
| mysql |
| performance_schema |
| sys |
+-----+
5 rows in set (0.09 sec)
```

Figura 4.8 Folosirea comenzilor MySQL

Pentru construirea unei imaginii pentru server am folosit un fișier Dockerfile care conține comenzile necesare pentru deployment sau mai bine zis codul necesar pentru construirea serverului.

```
1 FROM node:18
2
3 WORKDIR /usr/app
4
5 COPY package.json containerScript.sh ./
6
7 RUN chmod +x ./containerScript.sh
8
9 RUN apt-get update \
10     && apt-get install -y wget gnupg \
11     && wget -q -O - https://dl-ssl.google.com/linux/linux_signing_key.pub | apt-key add - \
12     && sh -c 'echo "deb [arch=amd64] http://dl.google.com/linux/chrome/deb/ stable main" >> /etc/apt/sources.list.d/google.list' \
13     && apt-get update \
14     && apt-get install -y google-chrome-stable fonts-ipafont-gothic fonts-wqy-zenhei fonts-thai-tlwg fonts-kacst fonts-freefont-ttf libxss1 \
15     --no-install-recommends \
16     && rm -rf /var/lib/apt/lists/*
17
18 RUN npm i
19
20 COPY . .
21
22 CMD ["./containerScript.sh"]
```

Cod 4.15 Fișierul Dockerfile

În fișierul Dockerfile specific că directoul de lucru este /usr/app, copiez fișierul package.json(conține pachete folosite de către server) și scriptul containerScript.sh în rădăcină containerului, ofer drepturi de execuție asupra scriptului, instalez pachetele necesare, rulez “npm install” ca se se instaleze toate pachetele din fișierul package.json, iar la final rulez scriptul containerScript.sh.

```
1  #!/bin/bash
2  echo -e "${SERVICE_SETTING}" > /usr/app/.env
3  npm run migrate
4  node index.js
```

Cod 4.16 Scriptul folosit pentru deploy

Acest script copiază variabilele de environment în fișierul `/usr/app/.env`, rulează migrările care crează tabelele din baza de date, iar în final rulează `index.js` care este fisierul de intrare al serverului.

4.4 Conexiunea cu baza de date

Pentru crearea conexiunii cu baza de date MySQL am folosit un ORM(Object Relation Mapping) numit Sequelize care ne ajută să facem operații asupra unei baze de date.

```
1  const Sequelize = require('sequelize');
2  const dotenv = require('dotenv');
3
4  dotenv.config();
5
6  const sequelize = new Sequelize(
7    process.env.DATABASE_NAME,
8    process.env.DATABASE_USER,
9    process.env.DATABASE_PASSWORD,
10    { host: process.env.HOST, dialect: process.env.DATABASE_DIALECT },
11  );
```

Cod 4.17 Conectarea la baza de date

Conexiunea la baza de date este destul de simplu implementată. După cum am spus, folosim librăria Sequelize care ne permite să creem un obiect de tip Sequelize care are următorii parametri în constructor:

1. Numele bazei de date preluat din variabila de mediu `DATABASE_NAME`
2. Numele utilizatorului preluat din variabila de mediu `DATABASE_USER`
3. Parola bazei de date preluată din variabila de mediu `DATABASE_PASSWORD`
4. Un obiect care specifică hostul și dialectul întrucât acest ORM se poate folosi pentru mai multe baze de date nu doar MySQL. În cazul acestui proiect dialectul este `mysql`. De asemenea, folosesc librăria `dotenv` care îmi permite să folosesc variabilele de mediu stocate în fișierul `.env` din rădăcină proiectului.

```
1  require('dotenv').config();
2
3  module.exports = {
4    development: {
5      username: process.env.DATABASE_USER,
6      password: process.env.DATABASE_PASSWORD,
7      database: process.env.DATABASE_NAME,
8      host: process.env.HOST,
9      dialect: 'mysql',
10   },
11   test: {
12     username: process.env.DATABASE_USER,
13     password: process.env.DATABASE_PASSWORD,
14     database: process.env.TEST_DATABASE_NAME,
15     host: process.env.HOST,
16     dialect: 'mysql',
17   },
18   production: {
19     username: process.env.DATABASE_USER,
20     password: process.env.DATABASE_PASSWORD,
21     database: process.env.PROD_DATABASE_NAME,
22     host: process.env.HOST,
23     dialect: 'mysql',
24   },
25 };
```

Cod 4.18 Configurarea datelor de conexiune la DB pentru diferite medii

După ce am creat obiectul Sequelize, tot ce rămâne de făcut este să importăm instanță în fișierul de intrare al serverului, iar conexiunea la baza de date este realizată.

```
55 //MySQL connection
56 require('./src/configs/sequelize/connection');
```

Cod 4.19 Realizarea conexiunii cu baza de date

Sequelize ne pune la dispoziție și o metodă prin care putem afișa dacă acțiunea de conectare la baza de date a avut success sau nu:

```
91 sequelize.authenticate().then(() => {
92   console.log('Connected to db successfully!');
93   app.listen(PORT, { hostname: () => {
94     //server starts listening for any attempts from a client to connect at
95     console.log(`Now listening on port ${PORT}`);
96   }});
97 });
```

Cod 4.20 Verificarea conexiunii la baza de date

Acest ORM ne pune la dispoziție tot felul de metode pentru efectuarea de query-uri asupra bazei de date. În codul de la Cod 4.21 se poate observa cum se folosește Sequelize pentru a aduce toate ordinele de deplasare în ordine descrescătoare după coloana “createdAt” și aplicând filtrele date ca și parametru funcției:

```
42 const BTFetchAll = (filters) => {  
43   return BT.findAll( options: {where: {...filters}, order: [['createdAt', 'DESC']] });  
44 }  
45
```

Cod 4.21 Folosire findAll pentru a aduce toate ordinele de deplasare

4.5 Crearea tabelelor folosind migrari

Sequelize-cli ne pune la dispoziție diverse comenzi care ne ajută să creem migrări, seedere etc. Comanda pentru a crea o migrare este:

```
npx sequelize-cli migration:generate --name migration-skeleton
```

Figura 4.9 Comanda pentru crearea de migrari

Sequelize ne permite să definim locația unde se vor afla fișierele rezultate în urma comenzilor puse la dispoziție:

```
1 const path = require('path');  
2 require('dotenv').config();  
3  
4 module.exports = {  
5   'config': path.resolve('src', 'configs', 'sequelize', 'config', 'config.js'),  
6   'models-path': path.resolve('src', 'models'),  
7   'seeders-path': path.resolve('src', 'configs', 'sequelize', 'seeders'),  
8   'migrations-path': path.resolve('src', 'configs', 'sequelize', 'migrations')  
9 }
```

Cod 4.22 Configurarea locației fișierelor create de sequelize

Configurările de mai sus trebuie să se afle într-un fișier numite .sequelizerc pentru că ORM-ul să știe să se uite în el și să le respecte. În Codul de mai sus observam că am configurat locul unde vreau să se afle fișierele de configurări, modelele, fișierele care conțin seedere și fișierele cu migrări. Am folosit librăria path pentru a forma path-urile. În urma fișierului de configurare a rezultat următoarea structura de fișiere:



Figura 4.10 Structura de fișiere rezultată în urma configurării Sequelize

De acum încolo mereu când o să creem o migrare aceasta vă fi automat create în `src/configs/sequelize/migrations`.

```
1  'use strict';
2  const {
3    DATABASE_TABLE_NAMES: {RECEIPTS},
4  } = require('../../../../utils/constants');
5
6  module.exports = {
7    up(queryInterface, Sequelize) {
8      return queryInterface.createTable(RECEIPTS, { attributes: {
9        id: {
10          type: Sequelize.INTEGER,
11          allowNull: false,
12          autoIncrement: true,
13          primaryKey: true,
14        },
15        amount: {
16          type: Sequelize.INTEGER,
17          allowNull: false,
18        },
19        provider: {
20          type: Sequelize.STRING,
21          allowNull: false,
22        },
23        document: {
24          type: Sequelize.INTEGER,
25          allowNull: false,
26        },
27        date: {
28          type: Sequelize.DATE,
29          allowNull: false,
30        },
31        description: {
32          type: Sequelize.STRING,
33          allowNull: false,
34        },
35        selectedFile: {
36          type: Sequelize.BLOB('long'),
```

```
57         allowNull: false,  
58     },  
59     selectedFileName: {  
60         type: Sequelize.STRING,  
61         allowNull: false,  
62     },  
63     createdAt: {  
64         type: Sequelize.DATE,  
65         allowNull: false,  
66     },  
67     updatedAt: {  
68         type: Sequelize.DATE,  
69         allowNull: false,  
70     },  
71     deletedAt: {  
72         type: Sequelize.DATE,  
73     },  
74 },  
75 });  
76 },  
77  
78 down(queryInterface, Sequelize) {  
79     return queryInterface.dropTable(RECEIPTS);  
80 },  
81 };
```

Cod 4.23 Migrarea pentru crearea tabelii Receipts

4.6 Implementarea unui debounce search pentru lista de angajati.

Căutarea unui angajat în lista de angajați folosește un debounce search pentru a crește performanța și pentru ca căutarea să se efectueze doar o singură dată, după ce user-ul termină de introdus numele angajatului și nu la fiecare literă tastată.

Exista doua tipuri de debounce:

1. Debounce normal: există un timp setat de programator la sfârșitul căruia se efectuează funcția. În momentul în care un utilizator apasă pe o tastă și modifică valoarea inputului, timpul este resetat. În acest mod funcția de căutare se va efectua o singură dată după ce utilizatorul termină de tastat și nu pentru fiecare literă schimbată.

2. Debounce throttling: este asemănător cu debounce-ul normal însă odată ce utilizatorul scrie ceva în input, timpul setat de programator începe să se scurgă, iar la final are loc căutarea. Diferență dintre cele doua este că la debounce throttling, timpul nu este resetat la fiecare tastare, așadar dacă de exemplu timpul setat este de o secunda, funcția de căutare va fi efectuată după o secunda indiferent că utilizatorul a terminat sau nu de tastat numele angajatului pe care îl caută.

Am ales să folosesc debounce normal, în acest mod funcția de căutare se apelează o singură dată, la final, când user-ul termină de tastat. Pentru asta am creat un custom hook.

```
1 import { useEffect, useState } from 'react';
2
3 export default function useDebounceValue(value, delay) {
4   const [debouncedValue, setDebouncedValue] = useState(value);
5
6   useEffect( effect: () => {
7     // Update debounced value after delay
8     const timer = setTimeout( callback: () => {
9       setDebouncedValue(value);
10     }, delay);
11
12     // Cancel the timeout if value changes (also on delay change or unmount)
13     // This is how we prevent debounced value from updating if value is changed within the delay period.
14     // Timeout gets cleared and restarted.
15     return () => {
16       clearTimeout(timer);
17     };
18   }, deps: [value, delay]);
19
20   return debouncedValue;
21 }
```

Cod 4.24 Custom hook pentru a efectua debounce search

Pentru implementarea custom hook-ului am creat un state numit `debouncedValue` care reține valoarea. De asemenea, am folosit și un `useEffect` care are în array-ul de dependențe `[value, delay]` (valoarea și întârzierea) ceea ce înseamnă că logica din callback o să se apeleze de fiecare dată când se modifică valoarea stării `value` sau valoarea lui `delay`. Înăuntrul callback-ului folosesc funcția `setTimeout` care primește ca parametrii o funcție și un timp în milisecunde care vă efectua funcția dată ca parametru după intervalul de timp specificat de `delay`. În final, se efectuează un `clearTimeout()` care împiedică ca valoarea stării `debouncedValue` să se modifice în cazul în care inputul a fost modificat. În acest mod, la fiecare modificare a inputului de search, se face `clearTimeout()`, valoarea nu este modificată pentru că nu se termină timpul specificat de `delay`, iar valoarea stării vă fi schimbată doar la final.

```
25 const debouncedEmployeeNameFilter = useDebounceValue(employeeNameFilter, delay: 400);
```

Cod 4.25 Folosirea custom hook-ului

Textfield-ul care efectuează căutarea este următorul:

```
54 <TextField
55   size='small'
56   value={employeeNameFilter}
57   onChange={(e :... ) => handleOnSearch(e)}
58   placeholder='Employee name search'
59   InputProps={{ className: classes.input }}
60   multiline
61 />
```

Cod 4.26 Text field folosit pentru debounce search

În acest mod, la onChange se apelează funcția handleOnSearch care setează filtrul cu valoarea din input:

```
31      const handleOnSearch = (e) => {  
32          setEmployeeFilter(e.target.value);  
33      };
```

Cod 4.25 Funcția pentru debounce search

În momentul în care se apelează linia 32 de cod de mai sus, se schimbă valoarea constantei declarată în Cod 4.25 și astfel se apelează următorul callback din useEffect care realizează filtrarea:

```
35      useEffect( effect: () => {  
36          if (!tableData) return;  
37  
38          if (!debouncedEmployeeNameFilter.length) {  
39              setEmployeeData(tableData);  
40              return;  
41          }  
42  
43          const filteredEmployees = tableData.filter((el) => {  
44              return el.employeeName  
45                  .toLocaleLowerCase()  
46                  .includes(debouncedEmployeeNameFilter.toLocaleLowerCase());  
47          });  
48  
49          setEmployeeData(filteredEmployees);  
50      }, deps: [debouncedEmployeeNameFilter]);
```

Cod 4.27 Logica pentru filtrarea listei de salariați

Variabila “tableData” conține datele inițiale din tabel, în cazul în care datele nu există returnăm pentru că nu avem ce căutare să facem. De asemenea, dacă lungimea lui debouncedEmployeeNameFilter este 0 înseamnă că inputul de căutare este gol și din acest motiv setez datele din tabela cu lista inițială. Pe acest if se va intra în momentul în care a avut loc o căutare, iar apoi utilizatorul șterge filtrul de căutare, în acest caz vrem să arătăm din nou lista inițială și nu cea filtrată. În final, dacă nici una din cele două condiții nu este satisfăcută are loc filtrarea care nu este case sensitiv. La linia 49, este setată starea ce conține lista de angajați care vă declanșă că componenta să își face update, iar în acest mod se vă modifica lista de angajați cu noua lista filtrată.

4.7 Implementarea exportului CSV pentru deconturi

Pe partea de UI, pentru implementarea exportului, am folosit un buton care la onClick apelează o funcție care face un request la backend unde este format CSV-ul, iar răspunsul serverului este descărcat pe client.

```
{enableExportCSV && user?.roles.includes(ADMIN) && (  
  <Button  
    style={{ height: 40 }}  
    color='primary'  
    variant='contained'  
    size='medium'  
    startIcon={<DownloadIcon />}  
    onClick={handleExportCSV}>  
    Export CSV  
  </Button>  
)}
```

Cod 4.28 Butonul Export CSV de pe frontend

Observăm că butonul este randat doar dacă flag-ul enableExportCSV este true, iar utilizatorul este un administrator, deci doar un admin poate să facă această acțiune. Flagul de enableExportCSV este necesar deoarece există două paginii care folosesc aceeași componentă reutilizabilă: pagina ce afișează ordinele de deplasare și pagina ce afișează deconturile. Pentru că exportul CSV este o funcționalitate specifică paginii cu deconturi avem nevoie de un flag care randează sau nu butonul de export.

```
161 const handleExportCSV = async () => {  
162   const queryParams = searchParams.get('searchQuery');  
163   const requestURL = `${endpoints.returns}/csv${  
164     queryParams ? `?searchQuery=${queryParams}` : ''  
165   }`;  
166  
167   try {  
168     setCsvLoading( value: true);  
169     const csvData = await fetch(requestURL);  
170     setCsvLoading( value: false);  
171     if (csvData && csvData.data) {  
172       const downloadLink = document.createElement( tagName: 'a' );  
173  
174       const blob = new Blob( blobParts: [ '\uffff', csvData.data ] );  
175       downloadLink.href = URL.createObjectURL( blob );  
176  
177       const downloadFileName = `Returns_${new Date().toISOString()}.csv`;  
178       downloadLink.download = downloadFileName;  
179  
180       document.body.appendChild( downloadLink );  
181       downloadLink.click();  
182       document.body.removeChild( downloadLink );  
183     }  
184   } catch ( e ) {  
185     setCsvLoading( value: false );  
186     console.error( e );  
187   }  
188 }
```

Cod 4.29 Funcția handleExportCSV

Pentru început, funcția respectivă face un request de tip GET la server și setează parametrii în query în cazul în care există. Variabila requestURL conține url-ul spre care se va face requestul de la linia 169. După ce răspunsul este obținut de la server se crează un element invizibil pe pagină care va declanșa descărcarea fișierului. După ce am creat elementul, se formează un obiect Blob care primește un array ce conține datele csv-ului și un UTF-8 header, iar apoi se formează un string care reprezintă blob-ul respectiv(`URL.createObjectURL(blob)`) ce va fi setat în atributul href al link-ului. În cele din urmă, specificăm numele fișierului care va fi descărcat, adăugăm link-ul de download în corpul documentului, facem click pe el ca să se declanșeze descărcarea, iar apoi scoatem elementul din corpul documentului.

De asemenea, la linia 168 setez starea de loading pe true ca să arăt utilizatorului un spinner până când toată logica este executată și fișierul este descărcat. La final sau în caz de eroare stare de loading este setată pe false. Această stare este pasată mai departe unei componente copil care va afișa un spinner dacă starea are valoarea true.

```
1 import React from 'react';
2 import { Backdrop, CircularProgress } from '@mui/material';
3
4 const BackDropCommon = ({ loading }) => {
5   return (
6     <Backdrop sx={{ color: '#fff', zIndex: (theme) => theme.zIndex.drawer + 1 }} open={loading}>
7       <CircularProgress color='inherit' />
8     </Backdrop>
9   );
10 };
11
12 export default BackDropCommon;
```

Cod 4.30 Componentă folosită pentru ecranul de loading

Pe partea de server, CSV-ul este generat folosind o librărie numită json2csv care primește doi parametri:

1. Un array de obiecte de tipul {label: string, value: string}
2. Un array cu valoarea atributelor

```
469     const CSVData = [];  
470  
471     for (let i = 0; i < returns.length; i++) {  
472         const returnEntry = returns[i];  
473  
474         const employeeBankAccount = await getOne( whereFilters: {employeeId: returnEntry.employeeId});  
475  
476         CSVData.push({  
477             returnId: returnEntry.id,  
478             employee: returnEntry.employee,  
479             bankAccount: employeeBankAccount?.bankAccount  
480         });  
481     }  
482  
483     const csvHeader = [  
484         {label: 'Return id', value: 'returnId'},  
485         {label: 'Employee name', value: 'employee'},  
486         {label: 'Bank Account', value: 'bankAccount'}  
487     ];  
488  
489     const generatedCSV = json2csv.parse(CSVData, {  
490         fields: csvHeader  
491     });  
492  
493     res.status(200).json(generatedCSV);
```

Cod 4.31 Generarea fișierului CSV de pe backend

Serverul, face un query în tabela de BankAccounts pentru fiecare decont și aduce datele pentru salariatul asociat decontului. Rezultatul este adăugat în arrayul CSVData folosind metoda push. Am declarat și o variabilă numită csvHeader care conține numele fiecărei coloane din fișier. În final, folosind metoda „parse” oferită de librărie, se formează fișierul CSV care este trimis ca și răspuns la client.

5. Utilizarea aplicației

5.1 Ecranul de login

Pagina de login ne permite să ne logăm în aplicație apăsând butonul de SSO login ce ne redirecțiază către API-ul de la Microsoft.

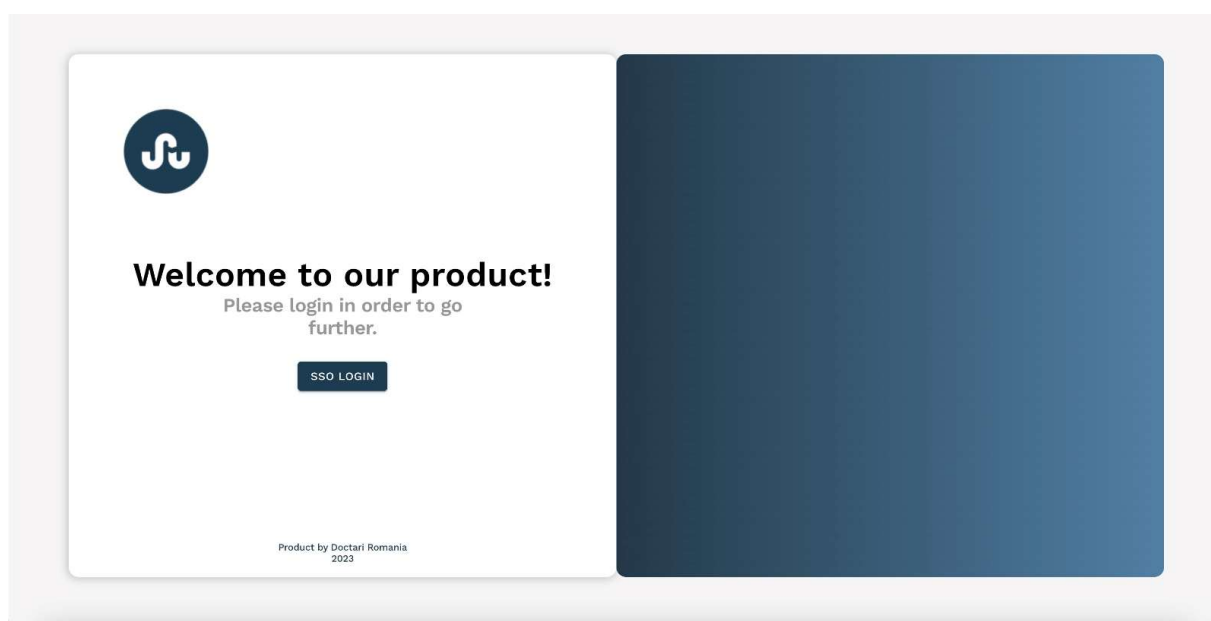


Figura 5.1 Ecranul de Login

5.2 Ecranul cu ordinele de deplasare

Aceasta este prima pagină către care suntem redirecțiați după login. De pe această pagină putem naviga pe pagina de deconturi, de angajați și de setări cu ajutorul tab-urilor din bara de navigare. În această pagină putem vedea lista cu ordinele de deplasare, putem accepta, refuza, șterge sau descărca actele pentru ordinul de deplasare. Intrările din tabel pot fi sortate după fiecare coloană din header, în momentul în care dăm hover peste numele unei coloane, va apărea o săgeată care ne permite sortarea crescătoare sau descrescătoare. De asemenea, există filtru de employee și de status care ne ajută să filtrăm rezultatele din tabel. Există operațiile de bulk accept, delete și reject dacă selectăm mai multe intrări din tabel dând click pe checkbox-ul din dreptul lor, sau pe cel din header că să selectăm toată pagina. De asemenea, se poate seta numărul de intrări per pagină, valoarea inițială de 5 intrări se poate schimba la 10 sau 25. Din această pagină putem vedea și notificările dacă dăm click pe iconiță cu clopoțel din dreapta sus în bara de navigare. Din această pagină ne putem schimba poza de profil dacă dăm click pe imaginea de avatar sau putem să ieșim din aplicație. De menționat că toate aceste acțiuni sunt vizibile dacă suntem administrator, în caz

contrar nu s-ar vedea taburile de Employees și Settings, am vedea doar ordinele noastre de deplasare și nu am putea face multe alte acțiuni.

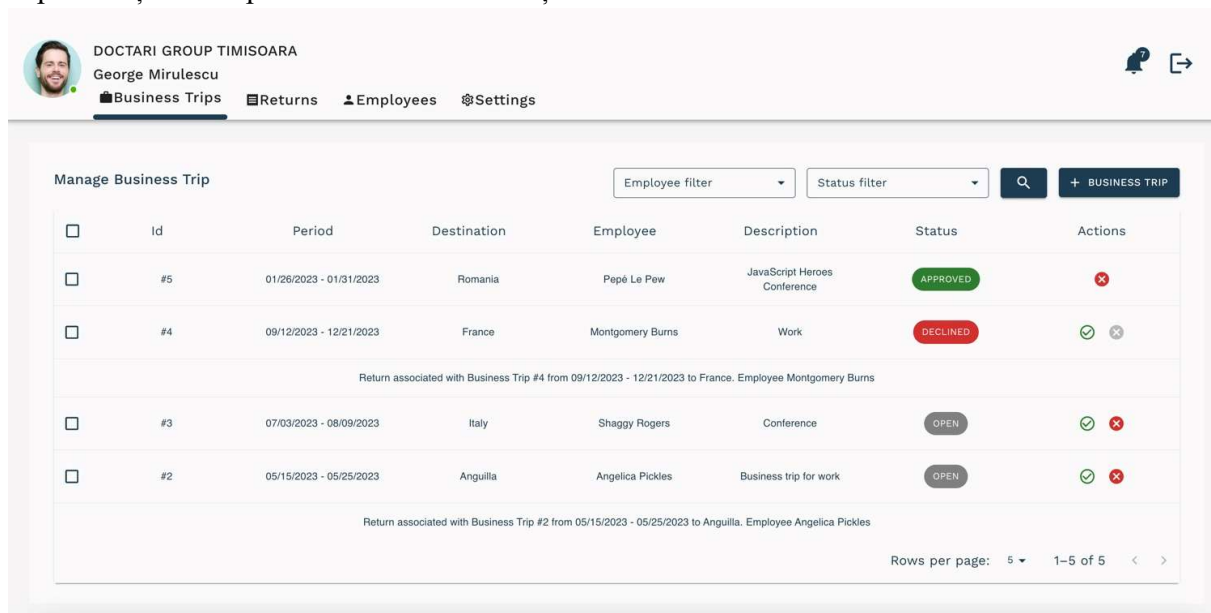


Figura 5.2 Ecranul de Business Trips

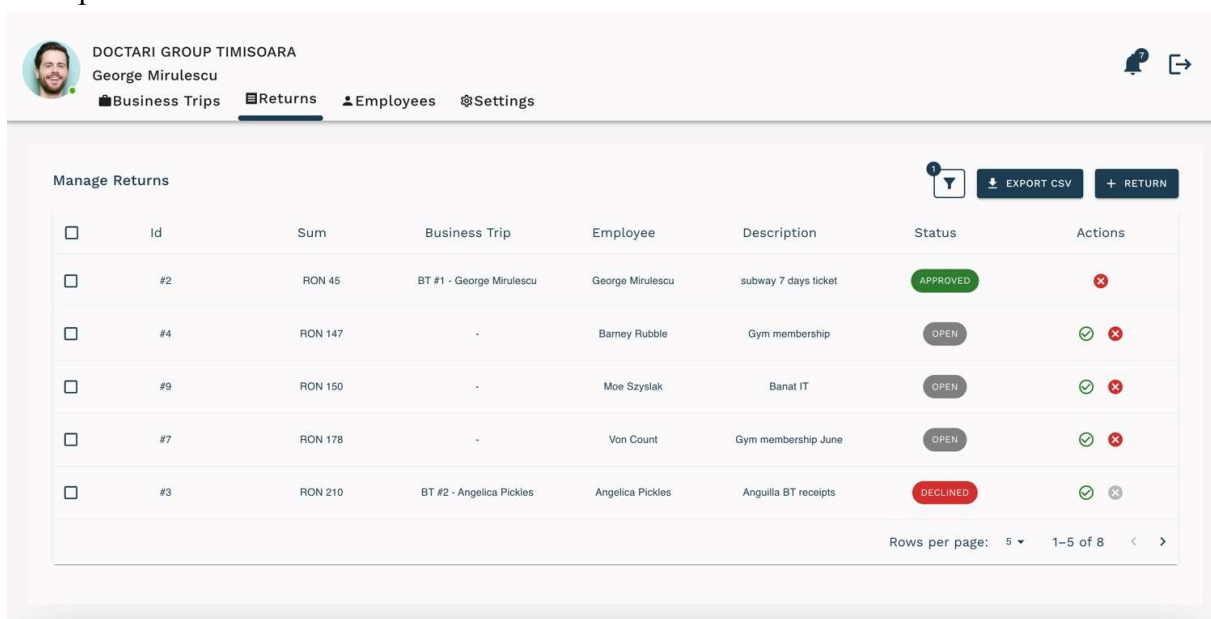
5.3 Ecranul în care putem crea un ordin de deplasare

Dacă apăsăm pe butonul de „Business Trip” din Figura 5.2 ajungem pe pagina pe care putem crea un ordin de deplasare:

Figura 5.3 Ecranul pentru crearea unui Business Trip

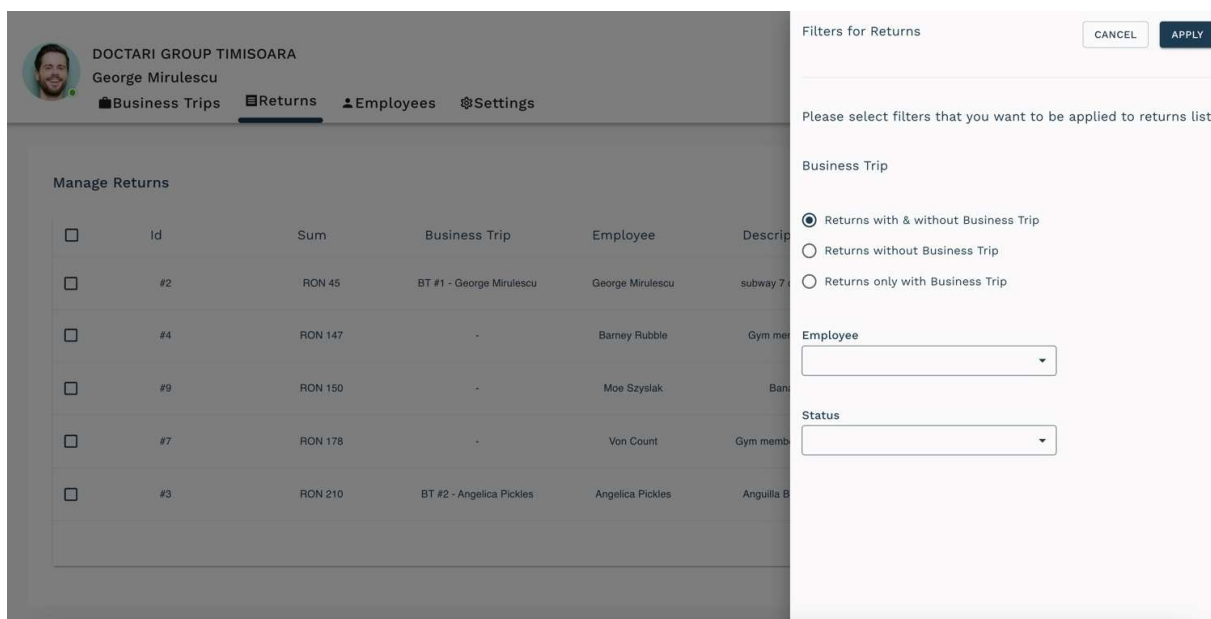
5.4 Ecranul cu deconturi

Acesta este asemănător cu ecranul ce afișează ordinele de deplasare pentru că folosește aceeași componentă reutilizabilă. Această pagină ne permite acțiuni asemănătoare celor de pe pagina cu ordine de deplasare. Deoarece există mai multe filtre, pagina de deconturi conține un drawer care poate fi accesat dacă apăsăm pe iconiță de filtre. O acțiune suplimentară oferită este butonul de Export CSV vizibil doar de admin.



Id	Sum	Business Trip	Employee	Description	Status	Actions
#2	RON 45	BT #1 - George Mirulescu	George Mirulescu	subway 7 days ticket	APPROVED	
#4	RON 147	-	Barney Rubble	Gym membership	OPEN	
#9	RON 150	-	Moe Szyslak	Banat IT	OPEN	
#7	RON 178	-	Von Count	Gym membership June	OPEN	
#3	RON 210	BT #2 - Angelica Pickles	Angelica Pickles	Anguilla BT receipts	DECLINED	

Figura 5.4 Ecranul de Returns



Filters for Returns

CANCEL APPLY

Please select filters that you want to be applied to returns list.

Business Trip

☒ Returns with & without Business Trip
☐ Returns without Business Trip
☐ Returns only with Business Trip

Employee

Status

Figura 5.5 Ecranul Cu Filtrele pentru Returns

5.5 Ecranul în care putem crea un decont

Dacă apăsăm butonul de „Return” din Figura 5.4 vom ajunge pe pagina în care putem crea un decont. Aceasta are doua taburi:

1. Tabul de General Details, în care selectăm angajatul, dacă nu suntem admin, inputul este disable și suntem automat selectați, asignăm un ordin de deplasare decontului (acest lucru nu este obligatoriu), introducem data, o descriere și semnăm utilizând mouse-ul.

2. Tabul de Receipts care ne permite să adăugăm bonuri fiscale decontului că să facem prezentă dovada plății. Se pot adăuga oricâte bonuri fiscale care sunt vizualizate în tabelul din dreapta care poate fi sortat crescător sau descrescător în funcție de orice coloana din header. Din nou, săgeata de sortare apare atunci când dam hover peste un header. De asemenea, dacă am adăugat un bon fiscal greșit îl putem șterge de tot apăsând pe iconiță cu X-ul roșu care vă deschide un modal în care să confirmăm acțiunea. Se poate și edita un receipt atunci când dam click pe o intrare din tabelul din dreapta, în urma click-ului form-ul din stangă este populat automat și putem edita bonul fiscal.

The screenshot shows a web application interface for creating a 'Return'. At the top, there's a header with a user profile (DOCTARI GROUP TIMISOARA, George Mirulescu) and navigation links (Business Trips, Returns, Employees, Settings). The 'Returns' tab is active. Below the header, the 'New Return' form is displayed. It has two tabs: 'General Details' (selected) and 'Receipts'. The 'General Details' tab contains several input fields: 'Employee' (a dropdown menu showing 'George Mirulescu'), 'Business Trip' (a dropdown menu showing 'None'), 'Create date a' (a date picker showing '18.05.2023'), 'Description' (a text area containing 'abonament sala'), 'Employee signature' (a signature field with a handwritten signature), and 'Adviser signature' (an empty signature field). At the bottom of the form, there are two buttons: 'CANCEL' and '+ CREATE'.

Figura 5.6 Tab-ul General Details din ecranul pentru crearea unui Return

New Return General Details **Receipts**

Amount
 Eg. 490 RON

Provider Name
 Eg. BANU Sport

Doc. Number
 Eg. 13

Date
 dd.mm.yyyy

Description
 Eg. This will be a description of the receipt

Document Image
 Press in order to select an image...

Amount	Provider	Document	Description
150,00 RON	Gym One 5	1	abonament sala

Receipts Total: 150,00 RON Total Amount: 150,00 RON

Figura 5.7 Tab-ul Receipts din ecranul pentru crearea unui Return

Dacă dorim să edităm un decont putem face click pe orice intrare din tabelul prezent în Figura 5.4. Acțiunea va deschide pagina de edit return care ne permite să edităm toate datele, iar pe lângă cele doua taburi de General Details și Receipts mai apare un al treilea tab în care putem vizualiza Pozele asociate fiecărui receipt.

Edit Return #13 General Details Receipts **Photos**

Amount	Provider	Date	Description
450,00 RON	AWS S.R.L.	2023-05-18	decontare curs AWS

Receipts Total: 450,00 RON Total Amount: 450,00 RON

Photo of receipt: RECEIPT OF SALE, SUBP'S NAME, 11/12/2020 11:13:04 AM, Total 32.1, Cash 49.0, Change 7.9, THANK YOU!

Figura 5.8 Tab-ul Photos din ecranul pentru editarea unui Return

5.6 Ecranul administratorilor în care se poate adaugă contul bancar pentru fiecare salariat

Această pagină ne permite să configurăm contul bancar pentru fiecare angajat. Această configurare este utilă la export CSV care conține și o coloană cu contul bancar al angajatului pentru că administratorul să știe în ce cont trebuie să plătească suma.

Employee	Bank Account
Daffy Duck	NL48RABO2122013192
George Mirulescu	RO67RZBR5968921417414429
Daria Morgendorffer	MC6630003000507237255462H92
Moe Szyslak	PL20109024028915464548966885
Angelica Pickles	ES7500752188526733614313

Figura 5.9 Ecranul de Employees

5.7 Ecranul de setări

Acesta conține 3 pagini între care se poate naviga folosind bara verticală din stangă. Prima pagină care conține setările de profil ne permite să schimbăm sau să ștergem poza de profil.

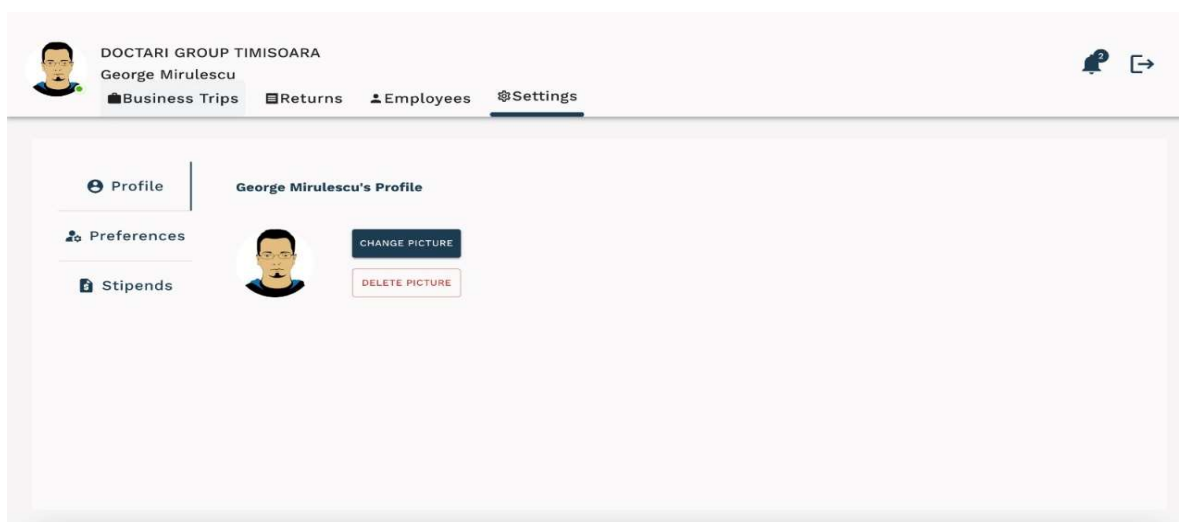


Figura 5.10 Ecranul de setări pentru profil

A doua pagină conține setările de preferință ale utilizatorului. Până în acest moment, există doar setarea de dark mode. În aceasta pagina se pot vizualiza și permisiunile(admin, normal user) care, spre deosebire de dark mode, nu se pot modifica.

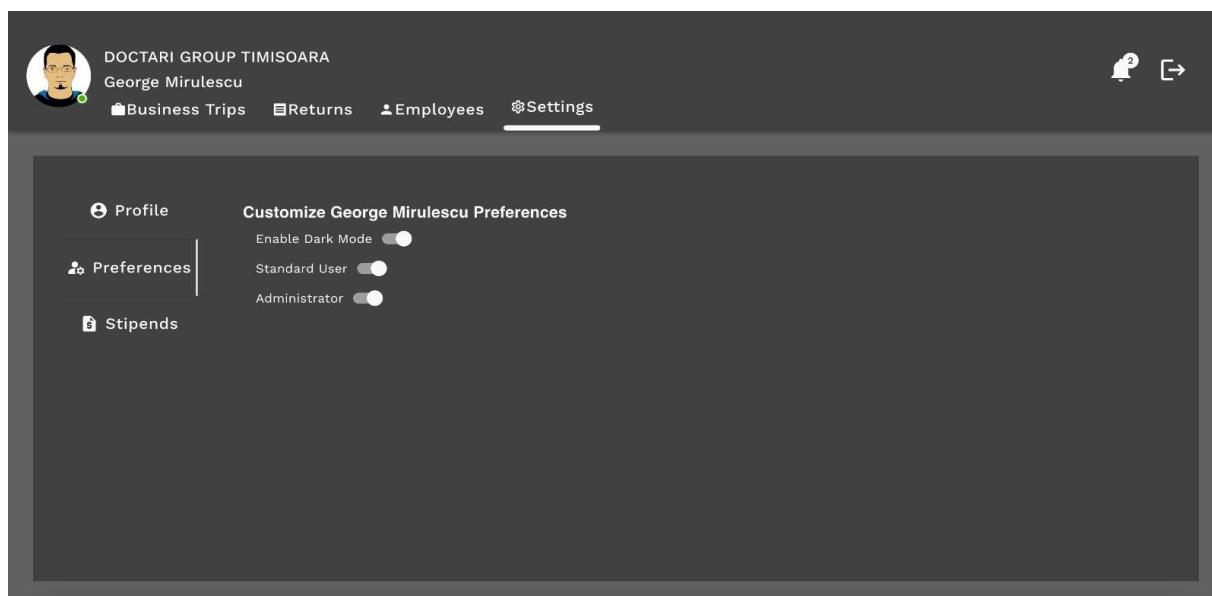
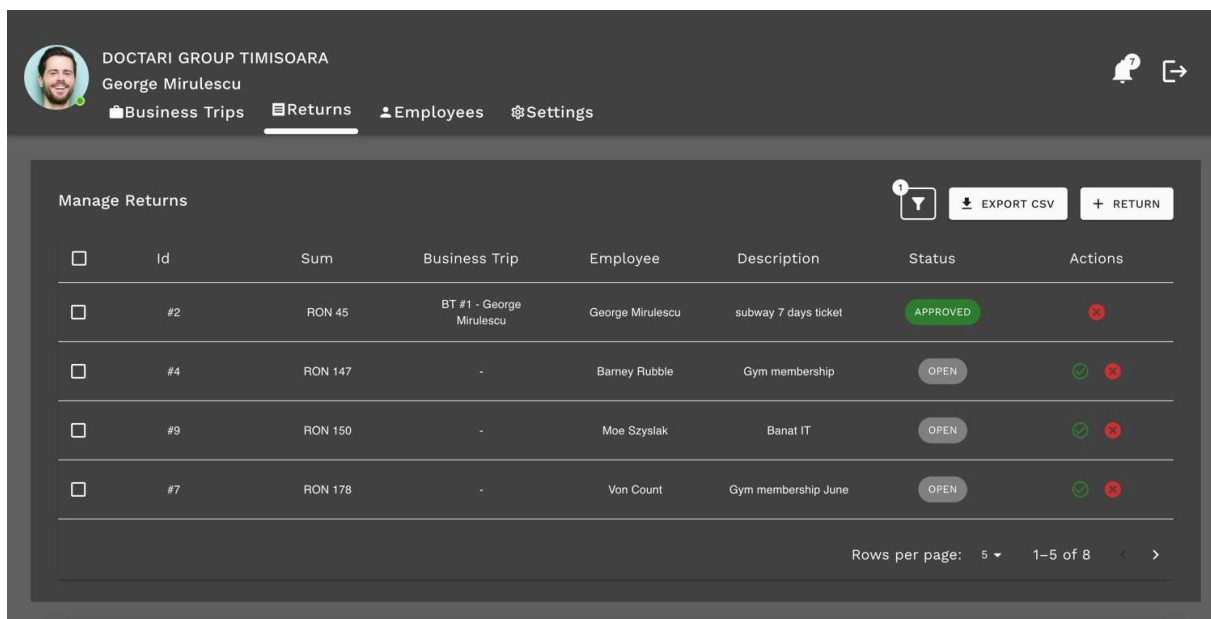


Figura 5.11 Ecranul de setări ale preferințelor

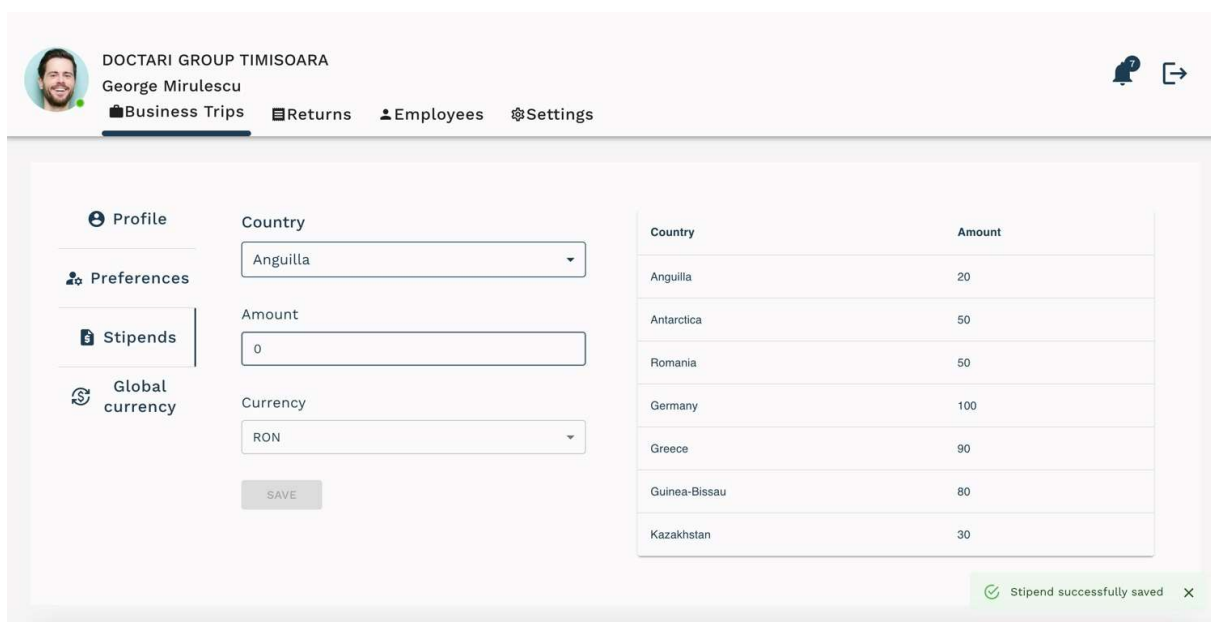
În momentul în care am activat dark mode se schimbă toată tematica aplicației:



Id	Sum	Business Trip	Employee	Description	Status	Actions
#2	RON 45	BT #1 - George Mirulescu	George Mirulescu	subway 7 days ticket	APPROVED	
#4	RON 147	-	Barney Rubble	Gym membership	OPEN	
#9	RON 150	-	Moe Szyslak	Banat IT	OPEN	
#7	RON 178	-	Von Count	Gym membership June	OPEN	

Figura 5.12 Ecranul de Returns în dark mode

Ultima pagină din setări este cea în care se pot configura diurnele pentru fiecare țară:



Country	Amount
Anguilla	20
Antarctica	50
Romania	50
Germany	100
Greece	90
Guinea-Bissau	80
Kazakhstan	30

Figura 5.13 Ecranul de configurare a diurnelor

6. Concluzii

6.1 Concluzii Generale

Aplicația Easy Returns este o aplicație care ajută firmele mici, mijlocii și mari să își gestioneze deconturile și ordinele de deplasare. Aceasta ne pune la dispoziție o interfață intuitivă, modernă cu foarte multe funcționalități ușor de folosit de către utilizator. Faptul că aplicația rezolvă problema birocrăției și reduce timpul folosit pentru a face un decont atât din perspectiva utilizatorului cât și din cea a administratorului face că aplicația să fie un adevrat ajutor pentru orice tip de firma.

Configurarea diurnelor pentru fiecare țară îl scutește pe administrator să se gândească pentru fiecare ordin de deplasare cât este diurna pe zi precum și de nevoia de a calcula manual bugetul total a unei deplasări. De asemenea, funcționalitățile de creare a ordinelor de deplasare și a deconturilor precum și generarea automată a hârtiilor legislative aferente acestora reduce foarte mult timpul irosit în mod normal.

Cel mai important aspect este că aplicația este ușor de întreținut, are un cost redus, iar mentenanța se poate realiza fără probleme datorită pipeline-urilor care ne oferă posibilitatea de CI/CD (continuous integration and continuous deployment). Datorită fișierelor .yaml codul de producție poate fi modificat și adaptat foarte ușor cu un simplu pull request din branch-ul dev în branchul main.

6.2 Funcționalități viitoare

EasyReturns este o aplicație cu un design modern și o grămadă de funcționalități care vine în ajutorul firmelor pentru gestionarea deconturilor și a ordinelor de deplasare. Aplicația oferă toate funcționalitățile de care are nevoie o firmă însă se mai pot implementa diverse funcționalități pentru a oferi o libertate mai mare și o configurare mai complexă în cazul în care este nevoie, crescând astfel gradul de utilitate al aplicației.

O primă funcționalitate ar fi să permitem utilizatorului să aleagă formatul fișierului în care sunt generate hârtiile asociate deconturilor și respectiv ordinelor de deplasare. În momentul de față, descărcarea actelor are loc doar în format PDF, însă poate cineva are nevoie să modifice puțin fișierul și ar fi util să permitem și alte tipuri de fișiere cum ar fi docx.

O a doua funcționalitate ar fi că administratorul să poată configura permisiuni pentru fiecare anagajat în parte. În momentul de față, un utilizator standard poate crea un ordin de deplasare sau un decont doar pentru el însuși. În cazul unui manager, poate ne dorim să poată să facă asta pentru toți oamenii din echipa lui, însă nu pentru toți angajații. De asemenea, poate am dori ca un manager să poată accepta sau refuza deconturi fără să mai fie nevoie de intervenția directorului (administratorului).

LISTA SECVENȚELOR DE COD

Figura 2.1 Exemplu Github Action.....	12
Figura 2.2 Ilustrare utilizare Terraform	13
Figura 2.3 Modificarile aduse unui fișier observate din interfața oferită de Github.....	14
Figura 3.1 Infrastructura sistemului Easy Returns.....	15
Figura 3.2 Diagrama UML	17
Figura 3.3 Navigarea în aplicație.....	18
Figura 3.4 Pagina cu ordine de deplasare	19
Figura 3.5 Tabela BankAccounts.....	20
Figura 3.6 Tabela BusinessTrips.....	21
Figura 3.7 Tabela Notifications	22
Figura 3.8 Tabela Receipts	23
Figura 3.9 Tabela Returns.....	24
Figura 3.10 Tabela Stipends	25
Figura 3.11 Tabela UserProfiles	26
Figura 4.1 Permisuni necesare pentru a accesa poza de profil a unui utilizator	28
Figura 4.2 Raspunsul Algoritmului de OCR.....	34
Figura 4.3 Containerele rezultate în urma comenzii docker-compose up	35
Figura 4.4 Containerele observate în terminal folosind docker ps	35
Figura 4.5 Comanda docker ps	36
Figura 4.6 Docker exec pentru a intra în container.....	36
Figura 4.7 Conectarea la baza de date folosind linia de comandă	36
Figura 4.8 Folosirea comenzilor MySQL	37
Figura 4.9 Comanda pentru crearea de migrari	40
Figura 4.10 Structura de fișiere rezultată în urma configurării Sequelize	41
Figura 5.1 Ecranul de Login	48
Figura 5.2 Ecranul de Business Trips	49
Figura 5.3 Ecranul pentru crearea unui Business Trip.....	49
Figura 5.4 Ecranul de Returns	50
Figura 5.5 Ecranul Cu Filtrele pentru Returns.....	50
Figura 5.6 Tab-ul General Details din ecranul pentru crearea unui Return.....	51
Figura 5.7 Tab-ul Receipts din ecranul pentru crearea unui Return	52
Figura 5.8 Tab-ul Photos din ecranul pentru editarea unui Return.....	52
Figura 5.9 Ecranul de Employees	53
Figura 5.10 Ecranul de setări pentru profil	54
Figura 5.11 Ecranul de setări ale preferințelor.....	54
Figura 5.12 Ecranul de Returns în dark mode	55
Figura 5.13 Ecranul de configurare a diurnelor	55

LISTA SECVENȚELOR DE COD

Cod 4.1 Permisunile cerute la login si URL-ul la care se face requestul	28
Cod 4.2 Configurare MSAL	28
Cod 4.3 Construire obiect MSAL	29
Cod 4.4 Functia signIn	29
Cod 4.5 Functia getBackendToken	29
Cod 4.6 Functia getTokenRedirect	29
Cod 4.7 Configurare librarie Passport	30
Cod 4.8 Rutele oferite de către server	30
Cod 4.9 Implementarea interfetei pentru scanarea unui bon fiscal	31
Cod 4.10 Functia choosePhotoToScan	31
Cod 4.11 Declararea unei referinte	32
Cod 4.12 Functia scanReceipt	32
Cod 4.13 Logica de extragere OCR de pe backend	33
Cod 4.14 definirea serviciilor din fisierul docker-compose.yml	35
Cod 4.15 Fisierul Dockerfile	37
Cod 4.16 Scriptul folosit pentru deploy	38
Cod 4.17 Conectarea la baza de date	38
Cod 4.18 Configurarea datelor de conexiune la DB pentru diferite medii	39
Cod 4.19 Realizarea conexiunii cu baza de date	39
Cod 4.20 Verificarea conexiunii la baza de date	39
Cod 4.21 Folosire findAll pentru a aduce toate ordinele de deplasare	40
Cod 4.22 Configurarea locației fișierelor create de sequelize	40
Cod 4.23 Migrarea pentru crearea tabelii Receipts	42
Cod 4.24 Custom hook pentru a efectua debounce search	43
Cod 4.25 Folosirea custom hook-ului	43
Cod 4.26 Text field folosit pentru debounce search	43
Cod 4.27 Logica pentru filtrarea listei de salariați	44
Cod 4.28 Butonul Export CSV de pe frontend	45
Cod 4.29 Funcția handleExportCSV	45
Cod 4.30 Componentă folosită pentru ecranul de loading	46
Cod 4.31 Generarea fișierului CSV de pe backend	47

LISTA DE TABELE

Tabela 1.1: Analiză comparativă între SocrateExpense, Iconta și Easy Returns

BIBLIOGRAFIE

- [1] Sequelize, Migrations and Seeds, <https://sequelize.org/docs/v6/other-topics/migrations/>, [Online; Accesat: 08.05.2023]
- [2] Microsoft, What is single sign-on in Azure Active Directory?, <https://learn.microsoft.com/en-us/azure/active-directory/manage-apps/what-is-single-sign-on>, [Online; Accesat: 08.05.2023]
- [3] Microsoft, Azure Active Directory Seamless single sign-on <https://learn.microsoft.com/en-us/azure/active-directory/hybrid/how-to-connect-sso>, [Online; Accesat: 08.05.2023]
- [4] Node.js, What is Node.js? The JavaScript runtime explained https://courses.cs.duke.edu/compsci290/current/readings/What_is_Node.js_InfoWorld.pdf, [Online; Accesat: 08.05.2023]
- [5] Github, Understanding GitHub Actions <https://docs.github.com/en/actions/learn-github-actions/understanding-github-actions>, [Online; Accesat: 09.05.2023]
- [6] Node.js, <https://nodejs.org/en> , [Online; Accesat: 09.05.2023]
- [7] React, <https://react.dev/> , [Online; Accesat: 09.05.2023]
- [8] Docker, <https://www.docker.com/resources/what-container/#:~:text=A%20Docker%20container%20image%20is,tools%2C%20system%20libraries%20and%20settings>, [Online; Accesat: 09.05.2023]
- [9] Mindee, https://mindee.com/?gclid=CjwKCAjw3ueiBhBmEiwA4BhspKVhN9iUdxcEWX3VMdOwNnOIHNTdXZ6lbyfVhX13z32FseLVUCYvfRoCvcwQAvD_BwE [Online; Accesat: 09.05.2023]
- [10] AWS, <https://aws.amazon.com/>, [Online; Accesat: 09.05.2023]

Nr. 1 / 19.06.2023

ADEVERINȚĂ

Prin prezenta se adeverește că studentul/a GEORGE MIRULESCU, de la
Universitatea Politehnica Timișoara, Facultatea de Automatică și Calculatoare, specializarea
CTI-RO, a realizat lucrarea de licență intitulată
EASY RETURNS, sub
îndrumarea MIHAI BURADA în cadrul firmei
DOCTARI GROUP.

Studentul/a GEORGE MIRULESCU primește acordul firmei noastre pentru a
folosi în cadrul prezentării lucrării de diplomă/licență/disertație aplicația
EASY RETURNS și datele aferente acesteia. Aplicația este realizată în
întregime / o parte RETURNS (detalierea părții realizate) de către
studentul/a GEORGE MIRULESCU.

Data,

19.06.2023

Nume și prenume
Semnătura,

MARIUS PENTER



DECLARAȚIE DE AUTENTICITATE A
LUCRĂRII DE FINALIZARE A STUDIILOR*

Subsemnatul MIRULESCU GEORGE - REMUS

legitimă cu CI seria T7 nr. 787980

CNP 5010118410039

autorul lucrării EASY RETURNS - APLICAȚIE DESKTOP PENTRU GESTIONAREA
DECONTURILOR ȘI ORDINELORE DE DEPLASARE DINTR-O FIRMA

elaborată în vederea susținerii examenului de finalizare a studiilor de
LICENȚĂ organizat de către Facultatea

DE AUTONATICĂ ȘI CALCULATOARE din cadrul Universității

Politehnică Timișoară, sesiunea Iunie 2023 a anului universitar

2022 - 2023, coordonator CONF. DR. ING. ALEXANDRU TOPÎRCEANU, luând în

considerare art. 34 din Regulamentul privind organizarea și desfășurarea examenelor de
licență/diplomă și disertație, aprobat prin HS nr. 109/14.05.2020 și cunoscând faptul că în
cazul constatării ulterioare a unor declarații false, voi suporta sancțiunea administrativă
prevăzută de art. 146 din Legea nr. 1/2011 – legea educației naționale și anume anularea
diplomei de studii, declar pe proprie răspundere, că:

- această lucrare este rezultatul propriei activități intelectuale;
- lucrarea nu conține texte, date sau elemente de grafică din alte lucrări sau din alte surse fără ca acestea să nu fie citate, inclusiv situația în care sursa o reprezintă o altă lucrare/alte lucrări ale subsemnatului;
- sursele bibliografice au fost folosite cu respectarea legislației române și a convențiilor internaționale privind drepturile de autor;
- această lucrare nu a mai fost prezentată în fața unei alte comisii de examen/prezentată public/publicată de licență/diplomă/disertație;
- În elaborarea lucrării am utilizat instrumente specifice inteligenței artificiale (IA) și anume _____ (denumirea) _____ (sursa), pe care le-am citat în conținutul lucrării/nu am utilizat instrumente specifice inteligenței artificiale (IA)¹.

Declar că sunt de acord ca lucrarea să fie verificată prin orice modalitate legală pentru confirmarea originalității, consimțind inclusiv la introducerea conținutului său într-o bază de date în acest scop.

Timișoara,

Data

19.06.2023

Semnătura

Mirulescu

*Declarația se completează de student, se semnează olograf de acesta și se inserează în lucrarea de finalizare a studiilor, la sfârșitul lucrării, ca parte integrantă.

¹ Se va păstra una dintre variante: 1 - s-a utilizat IA și se menționează sursa 2 – nu s-a utilizat IA